

The complete project guide

What DevPulse is, why it exists, how it differs from GitHub, and exactly how every part works - architecture, data, algorithms, security, frontend, deployment and testing.

Product

AI-powered GitHub activity dashboard

Stack

React 19 · Node 22 · Express 5 · PostgreSQL 16

Version

main @ fe109b4 + digest cap fix

Contents

1	Executive summary	3
2	Why this project exists	5
3	How DevPulse differs from GitHub	7
4	Feature tour	9
5	System architecture	13
6	Backend deep dive	16
7	Authentication, authorization and security	19
8	Data model and migrations	23
9	The GitHub sync engine	25
10	The analytics engine	28
11	Review analytics	31
12	Team view (multi-repository)	33
13	AI summaries	35
14	Weekly digest	36
15	The demo workspace	38
16	Frontend architecture	39
17	Design system and motion	43
18	Deployment and configuration	45
19	Testing, quality gates and CI/CD	48
20	API reference	51
21	Limits, trade-offs and roadmap	53
22	Glossary and repository map	55

1 Executive summary

DevPulse is a web application that connects to a person's GitHub account, reads the activity of their repositories (commits, pull requests and code reviews), stores a bounded copy in PostgreSQL, and turns it into answers to questions that a team lead or engineer actually asks: *How long do pull requests wait for a first review? Which ones are waiting right now? Is one person doing all the reviewing? How does this week compare with the last? Which repositories are busy, and which are stuck?* It can also write a plain-English sprint summary (with or without an AI model) and deliver a **weekly digest** to Slack or e-mail.

What it is

- A React 19 single-page dashboard (Tailwind v4, Recharts, Base UI) with ten screens.
- A Node 22 / Express 5 API that can run as a long-lived server *or* as a single Vercel serverless function.
- A PostgreSQL 16 database with five versioned SQL migrations.
- GitHub OAuth for sign-in; the user's own token is used for every GitHub call.
- Optional Gemini or OpenAI summaries, always with a free rule-based fallback.
- Docker image, GitHub Actions CI/CD, and a free-tier deployment recipe (Vercel + Neon).

What is different about it

- **Review latency:** median time to first review, a live "waiting for a first review" list, reviewer concentration.
- **Team view:** up to 15 repositories read as one set of numbers, computed from raw rows so they add up.
- **Weekly digest:** the numbers are pushed to Slack/e-mail instead of waiting for someone to open a dashboard.
- **Honesty:** when GitHub's pagination hides history, DevPulse says so and withholds misleading comparisons.
- **Zero-setup demo:** a generated workspace that runs through the *real* pipeline with no credentials.

Dimension	Size / value
Backend	Node 22, Express 5, CommonJS, pg ; 10 service modules, 7 controllers, 5 migrations; 98 automated tests
Frontend	React 19, Vite 8, Tailwind v4, Recharts; 10 routed pages, ~60 components; 81 unit tests, 59 end-to-end/accessibility tests
Data kept per repository	up to 180 days of commits, pull requests and reviews (pruned at 210 days); no source code, ever
API surface	22 endpoints (auth, repositories, analytics, team, AI, digest, cron, health)
Deployment shapes	Vercel + Neon (free), or a single Docker container (Railway or any host)
Licence of the idea	University minor project (UPES synopsis: "AI-Powered GitHub Activity Dashboard"), extended to production quality

How to read this document

Chapters 2-4 explain *why* the project exists and how it differs from GitHub. Chapters 5-9 are the architecture (system, backend, security, data, sync). Chapters 10-15 explain every calculation and feature (analytics, reviews, team, AI, digest, demo). Chapters 16-20 cover the frontend, design system, deployment, testing and the API. Chapter 21 is an honest list of limits and roadmap, and chapter 22 a glossary and file map.

2 Why this project exists

2.1 The problem

Software teams generate a continuous stream of activity on GitHub. The raw facts are all there, but they are scattered: one repository per page, one pull request per page, one notification at a time. Nobody *sees* the shape of the work. Three concrete symptoms motivate DevPulse:

1. **Work waits invisibly.** A pull request that nobody has reviewed looks exactly like one that is being reviewed — until it is three weeks old. Delay between "ready" and "first look" is usually the largest, least visible part of cycle time.
2. **Load concentrates silently.** In most teams two or three people do most of the reviewing. That is a bottleneck and a bus-factor risk, but no GitHub screen shows reviewer distribution.
3. **Information is pull, not push.** Even where the data exists, someone must remember to look. Leads rely on gut feeling or an ad-hoc spreadsheet on Monday morning.

2.2 The goal of the original synopsis

The course synopsis asked for an *AI-powered GitHub activity dashboard*: GitHub OAuth login, a backend that collects commit and pull-request data, PostgreSQL storage, charts, an AI-written productivity summary, Docker and CI/CD. DevPulse implements all of that, then goes further on the two things that make such a tool worth using day to day: **depth** (review latency, reviewer load, size vs. speed) and **reach** (a team view and a push digest).

2.3 Honest framing

A fair question — and one that was asked during development — is: *"Isn't all of this already on GitHub?"* For the **commodity** charts (commits over time, contributor counts, a contribution calendar) the answer is largely yes, and DevPulse does not pretend otherwise. Those charts exist here as *context* and as the foundation the new analyses stand on. The reason to use DevPulse is the set of things GitHub's own repository Insights do not compute: time to first review, who is waiting now, reviewer concentration, size-versus-speed, a cross-repository view, and a scheduled digest. Chapter 3 spells this out feature by feature.

Design principle

Every number on screen must be explainable in one sentence and reproducible from data the user can see on GitHub. No opaque "productivity score", no surveillance-style per-person ranking by default, and no claim the data cannot support.

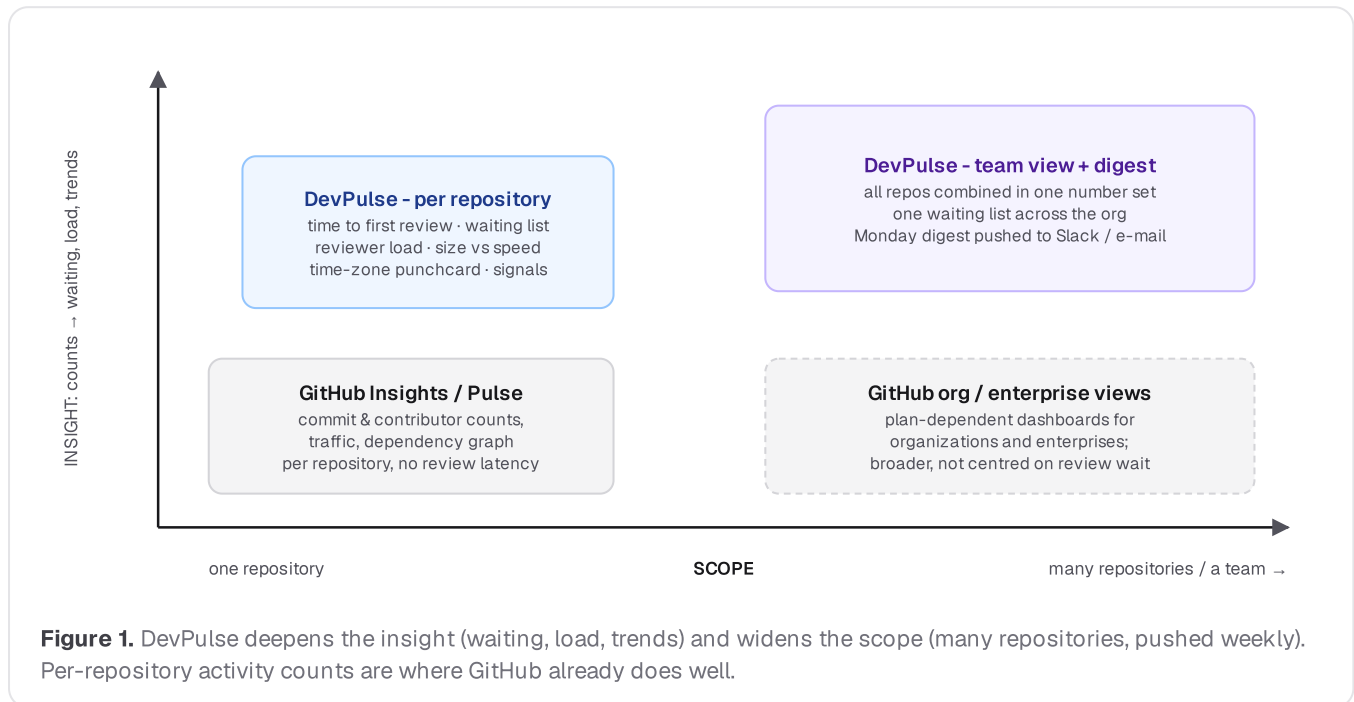
2.4 Who it is for

Persona	Question they bring	Where DevPulse answers it
Engineering lead / manager	Where is work stuck? Is review load healthy?	Reviews page, Team view, weekly digest

Persona	Question they bring	Where DevPulse answers it
Developer waiting on a review	Which of my PRs has nobody looked at yet?	Pull requests ("No review yet" badge), Overview waiting card
Tech lead of several repos	How do repositories compare this month?	Team page comparison table
Student / evaluator	Does it work end to end without my credentials?	One-click demo workspace
Open-source maintainer	Is activity trending up or down? Who carries reviews?	Overview deltas, Contributors, Reviewer load

3 How DevPulse differs from GitHub

GitHub is the system of record: it is authoritative for what happened. DevPulse is a *derived view*: it re-reads that record, keeps a short window of it, and computes things GitHub's own screens do not. This chapter is deliberately even-handed.



3.1 Feature-by-feature

Capability	GitHub (repository Insights and PR pages)	DevPulse	Verdict
Commit / contributor charts	Yes (Pulse, Contributors, Commit activity)	Yes: daily/weekly series with previous-period overlay, per-day top contributors in the tooltip	Parity - context only
Time zone of day/hour charts	Single repository-wide view	Punchcard and "busiest day" in the viewer's time zone (<code>tzOffset</code>)	Small plus
Time to merge	Not summarised; you read individual PRs	Median, p90, histogram, trend vs. previous period	Plus
Time to first review	Not computed	Median, p90, histogram, "reviewed within a day %", trend	New
Which PRs are waiting for a first review	Filter by <code>review:none</code> inside one repository	One ranked list, longest wait first, across all chosen repositories	New
Reviewer load / concentration	Not shown	Reviews per person, share, approvals vs. change requests, median response; "top reviewer share" signal	New

Capability	GitHub (repository Insights and PR pages)	DevPulse	Verdict
PR size vs. speed	Not shown	Merge time by XS/S/M/L/XL bucket and a plain-English insight	New
Several repositories together	Per repository (org/enterprise views are plan-dependent and broader)	Team page: combined KPIs, sortable comparison table, shared waiting list	New for individuals
Scheduled summary	Scheduled reminders can ping Slack with a list of open PRs awaiting review	Weekly digest: totals, trend, waiting list, top reviewers, optional AI paragraph; Slack and/or e-mail	Overlaps in part; digest adds trends
Plain-English health signals	No	Review speed, first review, review load, stale PRs, knowledge spread, work rhythm, momentum - each rated and explained	New
AI sprint summary	No (code-level Copilot features only)	Aggregate numbers only → insights and suggestions; rule-based fallback is free	New
Data ownership	n/a	Delete account removes everything and revokes the OAuth grant	Trust feature

3.2 What GitHub does better

- **Authority and completeness.** GitHub has every event since the repository was created. DevPulse keeps at most 180 days and, for very busy repositories, at most the 1,000 most recent commits and 500 most recently updated PRs.
- **Real-time.** GitHub is live. DevPulse is request-driven with a two-minute cache; a "Refresh" button forces a delta sync.
- **Depth on a single item.** Diffs, discussions, checks, timelines. DevPulse never reads code or comment bodies.
- **Native permissions.** DevPulse delegates authorization to GitHub (see chapter 7), which is the right place, but it means DevPulse can only show what the signed-in user can see.

3.3 Why not just use a third-party engineering-analytics product?

Commercial tools exist, are typically priced per developer, require org-wide installation, and ask for broad access. DevPulse is built to be adopted by *one person* in a minute (GitHub sign-in, no admin approval beyond the user's own repositories), to be self-hostable for free, and to be small enough to read end to end. It trades breadth (no DORA/deployment data yet, no issue tracker integration) for clarity and cost.

Not a replacement for judgement

Review-latency metrics describe flow, not individual performance. DevPulse shows reviewer *distribution* to expose bottlenecks, not to rank people, and the UI wording is chosen accordingly.

4 Feature tour

Ten routed screens, one command palette, one landing page. Each subsection says what the screen shows, which question it answers and where the numbers come from.

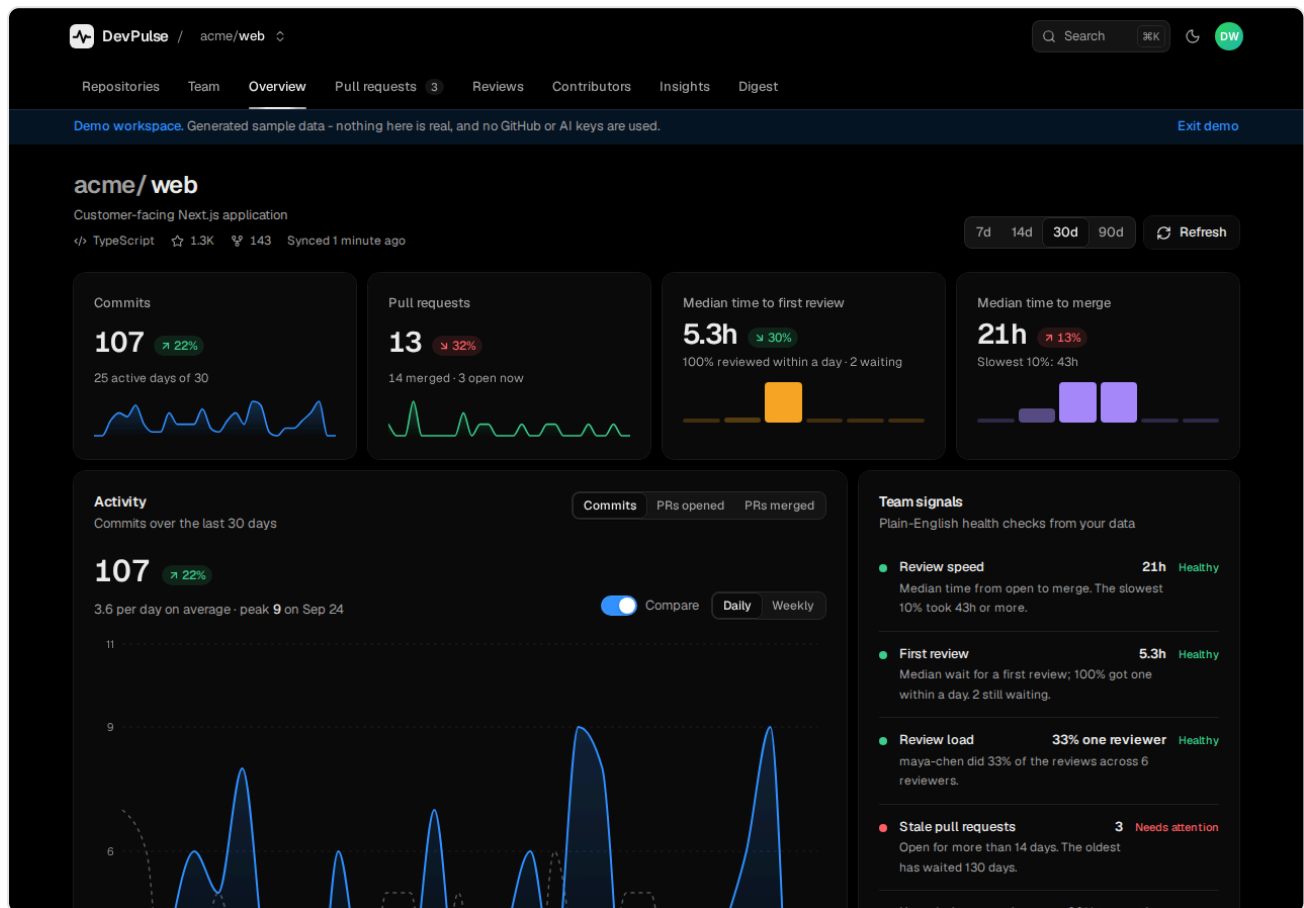


Figure 2. Overview (dark theme, demo data). KPI cards, interactive activity chart with previous-period overlay, health signals, calendar, punchcard, review queue and AI summary.

4.1 Landing page (signed out)

A marketing page with real motion: hero with floating "chips" that follow the pointer (parallax), an auto-scrolling product tour built from real screenshots of the demo workspace, feature rows, an AI summary that writes itself word by word, trust/limits section and FAQ. It respects `prefers-reduced-motion`. The primary call to action is **Explore the live demo**, which needs no account.

4.2 Repositories (portfolio)

Every repository the user can access, with search, sorting (recent / most active / stars / name), an "analyzed only" filter and a 30-day sparkline plus open-PR count for repositories already synced. Totals across analyzed repositories sit at the top. The list is cached server-side for five minutes; **Refresh list** forces a GitHub round trip. Opening a card selects the repository and goes to its Overview.

4.3 Team (portfolio)

Choose up to 15 repositories (stored in the URL as `?repos=a/b,c/d`). The page shows combined KPIs, an activity chart, health signals, a **sortable comparison table** (commits with delta and sparkline, merged, open and stale, time to merge, time to first review, waiting), one cross-repository waiting list, combined reviewer load and contributors. Repositories that cannot be read are listed as *left out* with the reason; very busy repositories raise an "incomplete history" banner.

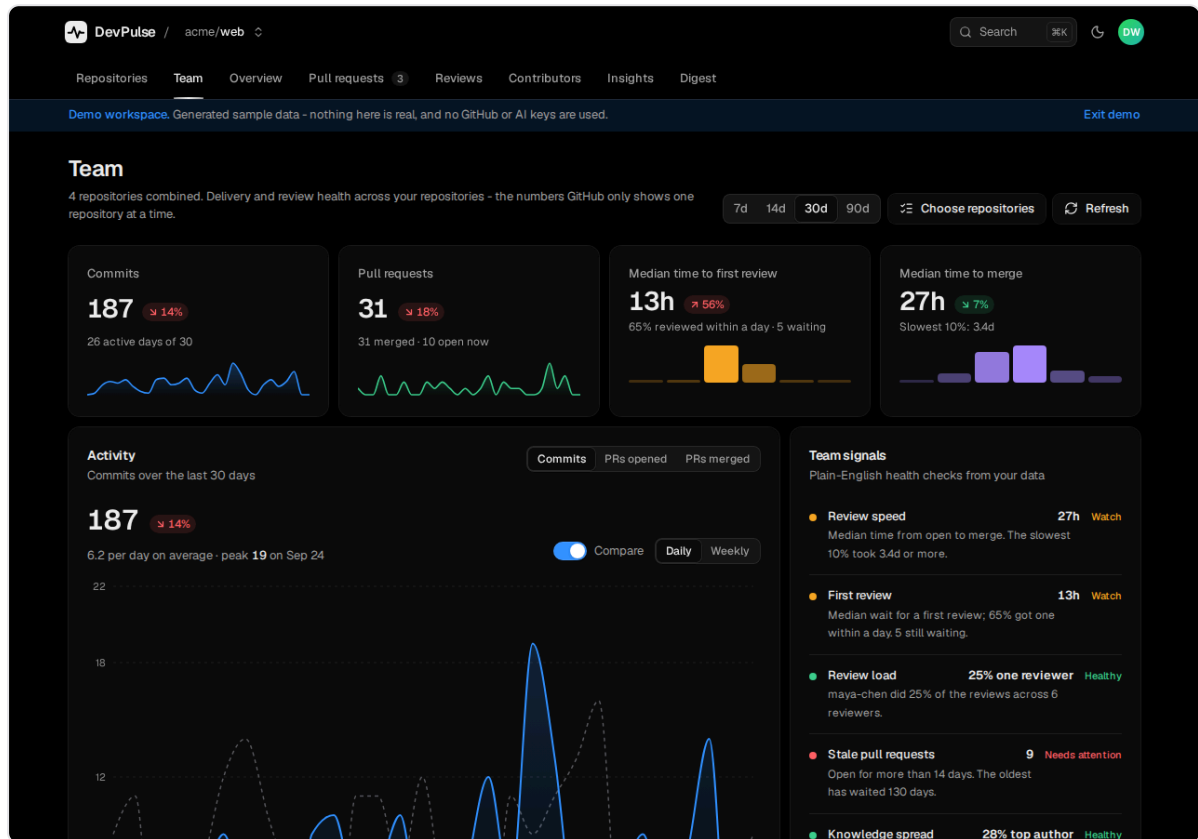


Figure 3. Team view: combined numbers on top, repositories compared below, one shared waiting list.

4.4 Overview (per repository)

Four KPI cards (commits, pull requests, **median time to first review**, median time to merge) each with a sparkline or histogram and a period-over-period delta. Below: the interactive **Activity** chart (commits / PRs opened / PRs merged; daily or weekly; compare with previous period; per-day top contributors in the tooltip; filter by contributor), **health signals**, contribution calendar, weekday × hour punchcard in the viewer's time zone, the lead-time histogram, the **waiting-for-review** card, contributors and the latest AI summary.

4.5 Pull requests

Searchable, filterable list (All / Open / Merged / Closed with counts), sortable by age, with badges: *Stale* (open > 14 days), *Draft*, *No review yet*, "merged in X", "X to review". Paginated 20 at a time.

4.6 Reviews

The page GitHub does not have. Four tiles - median first review (with trend), reviewed within a day %, waiting now, top reviewer share - then the waiting list, the first-review histogram, reviewer load (reviews, share, approvals, changes requested, median response) and a size-versus-merge-time chart with a generated sentence (e.g. "Large pull requests merge 3.1x slower than small ones"). Chapter 11 defines each number precisely.

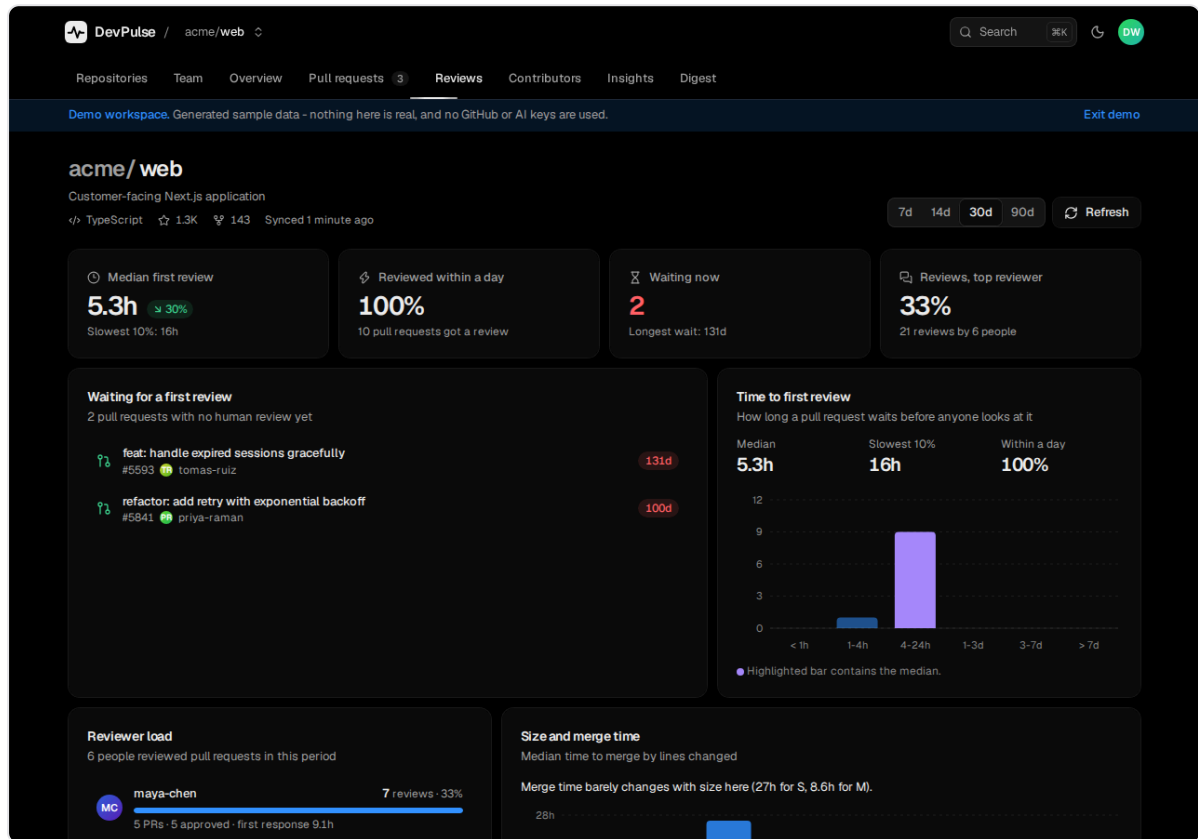


Figure 4. Reviews page: first-review speed, who is waiting, reviewer load and size versus merge time.

4.7 Contributors

Top contributors by commits and PRs with share, daily sparkline and PR count. Selecting a person filters the Overview chart and calendar to their work (`?who=login`). Also shows recent commits with copy-SHA buttons. Commit authors whose e-mail is not linked to a GitHub account are shown by name and clearly marked as unlinked.

4.8 Insights

Plain-English **signals** (derived with no AI) and the **AI sprint summary** history. Generating a report is rate- and quota-limited for paid providers; the rule-based provider is unlimited.

4.9 Weekly digest

Settings (repositories, weekday, Slack webhook, e-mail on/off, include AI, enabled) with a live preview in three formats: the exact HTML e-mail in a **sandboxed iframe**, a faithful Slack Block Kit rendering, and plain text. "Send test now" delivers it immediately. The demo can preview but not send.

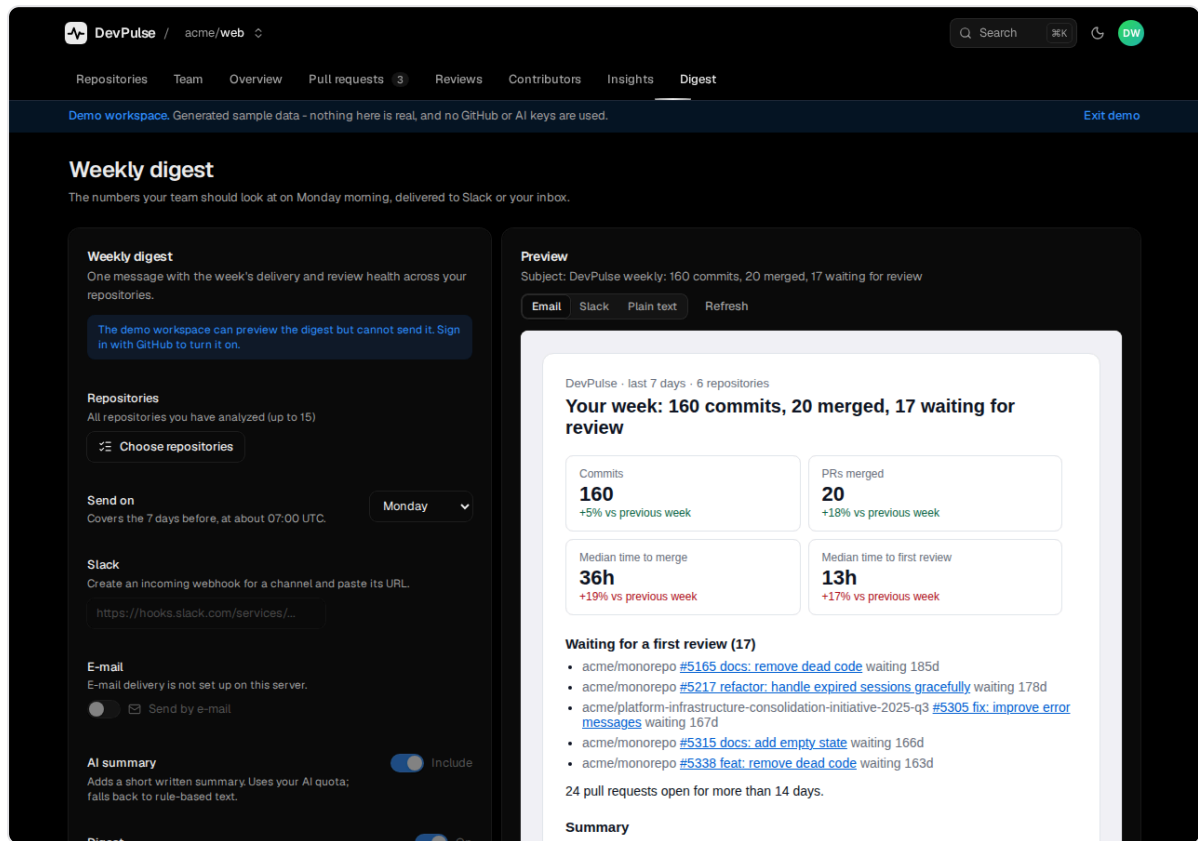


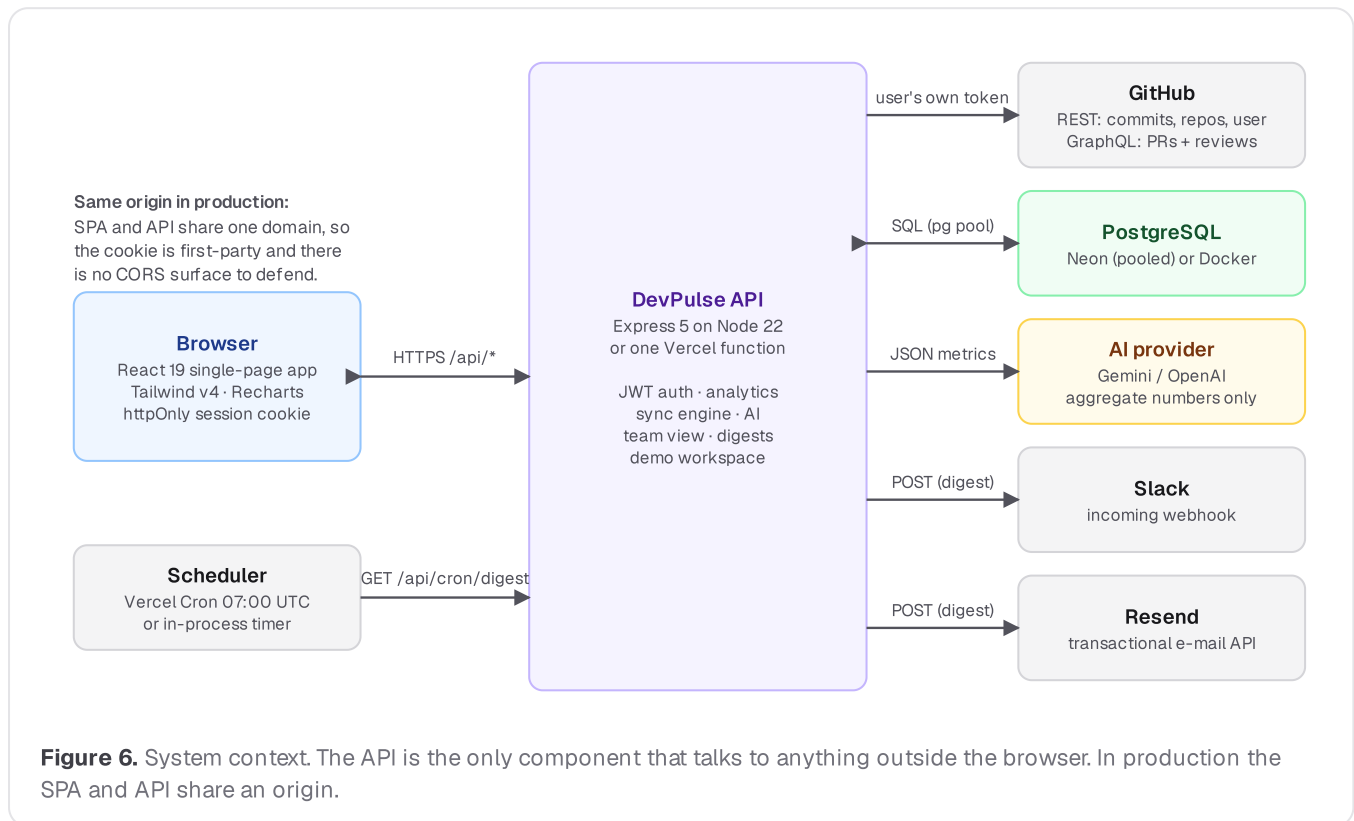
Figure 5. Digest settings with e-mail preview.

4.10 Cross-cutting UX

- **Command menu** (`Ctrl/Cmd+K`): jump to any repository or page, change time range, refresh, generate a summary, toggle theme, sign out. Opened from the keyboard it appears instantly (no animation) because keyboard actions repeat dozens of times a day.
- **Shareable URLs**: repository, range, contributor filter and team selection live in the query string, so Back works and a link reproduces the view.
- **Themes**: light, dark or system, applied before first paint by a tiny external script (`theme-bootstrap.js`) so there is no flash and the strict CSP is not weakened.
- **States**: every data view has designed skeleton, empty and error states; stale numbers dim while new ones load instead of flashing blank.
- **Accessibility**: WCAG 2.1 AA verified by axe-core on every page in both themes; charts expose text alternatives; animated numbers also exist as screen-reader text.
- **Account control**: "Delete account & data" requires typing the username and revokes the GitHub grant.

5 System architecture

Four moving parts - browser, API, database, GitHub - plus three optional outbound integrations (AI, Slack, e-mail) and one scheduler.



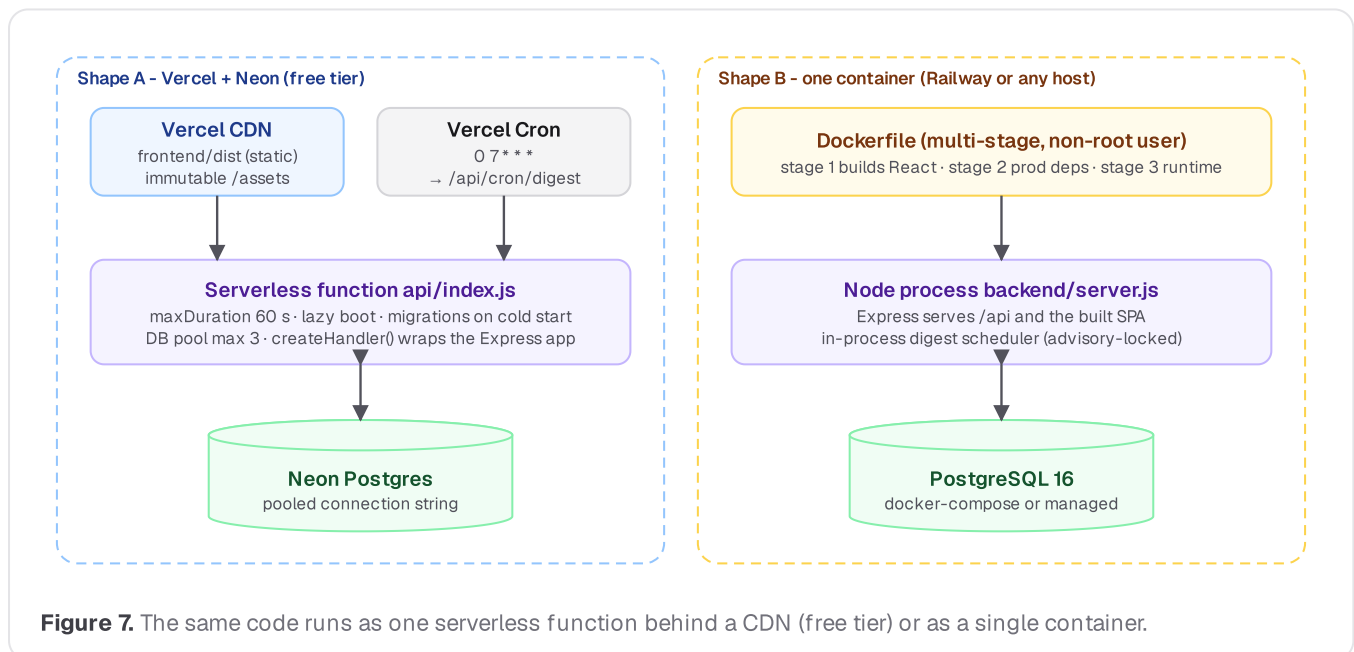
5.1 Responsibilities

Component	Owns	Does <u>not</u> do
Browser SPA	Rendering, interaction, URL state, theme, animation, client-side filtering/sorting of already-loaded data	Talk to GitHub or the AI provider directly; hold any secret; compute the heavy analytics
API	Authentication, authorization, GitHub access, sync, analytics, team aggregation, AI orchestration, digests, validation, rate limiting	Serve user-specific data without a verified session; store source code
PostgreSQL	Per-user copy of recent activity, encrypted credentials, AI reports, digest settings, migration state	Business logic (only constraints, upserts and indexes)
GitHub	Source of truth for activity and for who may see what	Know about DevPulse beyond the OAuth grant
AI provider	Turn a small JSON of <i>aggregate</i> metrics into prose	See commit messages, PR titles, code or names beyond top-contributor logins
Scheduler	Wake the digest runner daily	Decide who is due (the runner does)

5.2 Why these technology choices

Choice	Reason
Node + Express	Same language as the frontend, tiny surface, trivially wrapped as a serverless handler.
PostgreSQL (not a document store)	The data is relational (user → repo → commits/PRs/reviews); upserts with <code>ON CONFLICT</code> , partial indexes and advisory locks are exactly what the sync and digest claim need.
SQL migrations (not an ORM)	Five small files, explicit, reviewable, idempotent; applied under a Postgres advisory lock so two replicas cannot race.
React + Vite	Fast builds, code splitting by route, mature ecosystem for accessible primitives (Base UI) and charts.
Tailwind v4 with CSS variables	Design tokens (colours, easing, radii) defined once; light/dark is a variable swap, not a second stylesheet.
Recharts + hand-rolled SVG	Recharts for axes/tooltips on the main chart; custom SVG for sparklines, heatmap and punchcard where precise control matters.
GraphQL for pull requests	One call returns 50 PRs <i>with</i> their reviews and size; REST would need one extra call per PR.
httpOnly cookie JWT (not localStorage)	JavaScript on the page can never read the session, which removes a whole class of token theft.

5.3 Deployment shapes



The only code that differs between shapes is the entry point: `backend/server.js` (listen, in-process digest timer, graceful shutdown) versus `backend/serverless.js` (`createHandler`: lazy boot on first request, migrations on cold start under the advisory lock, failed boots retried and answered with 503 meanwhile). Everything below the entry point is shared.

5.4 Lifecycle of one dashboard view

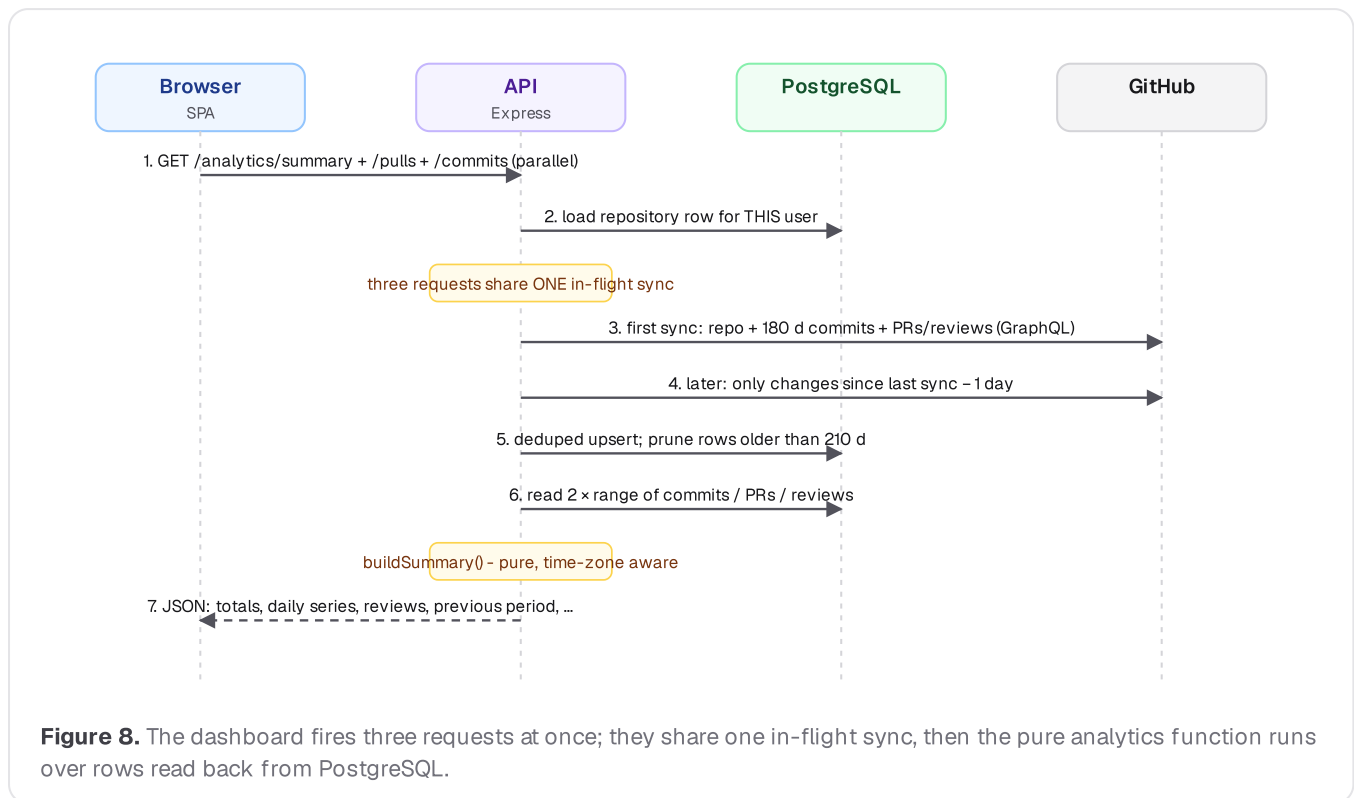
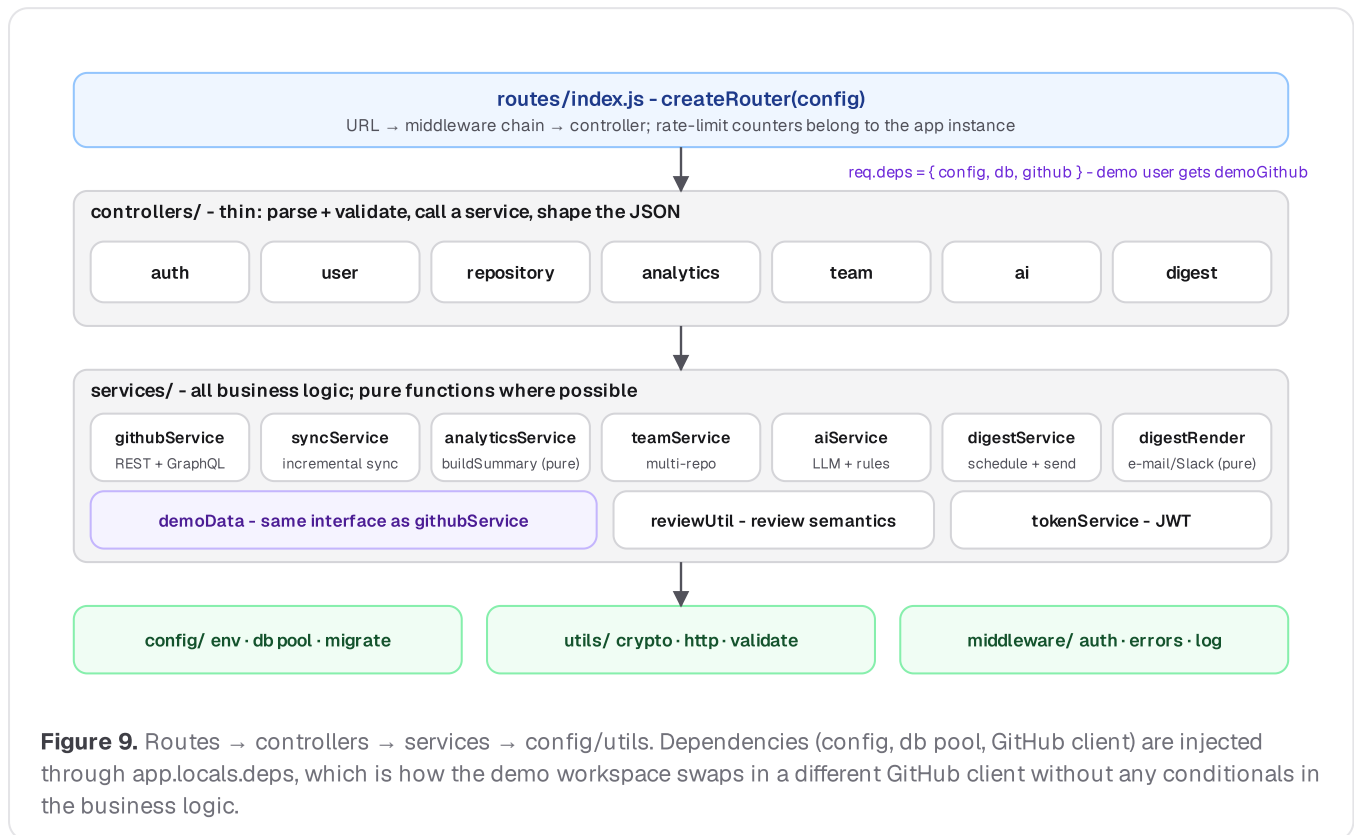


Figure 8. The dashboard fires three requests at once; they share one in-flight sync, then the pure analytics function runs over rows read back from PostgreSQL.

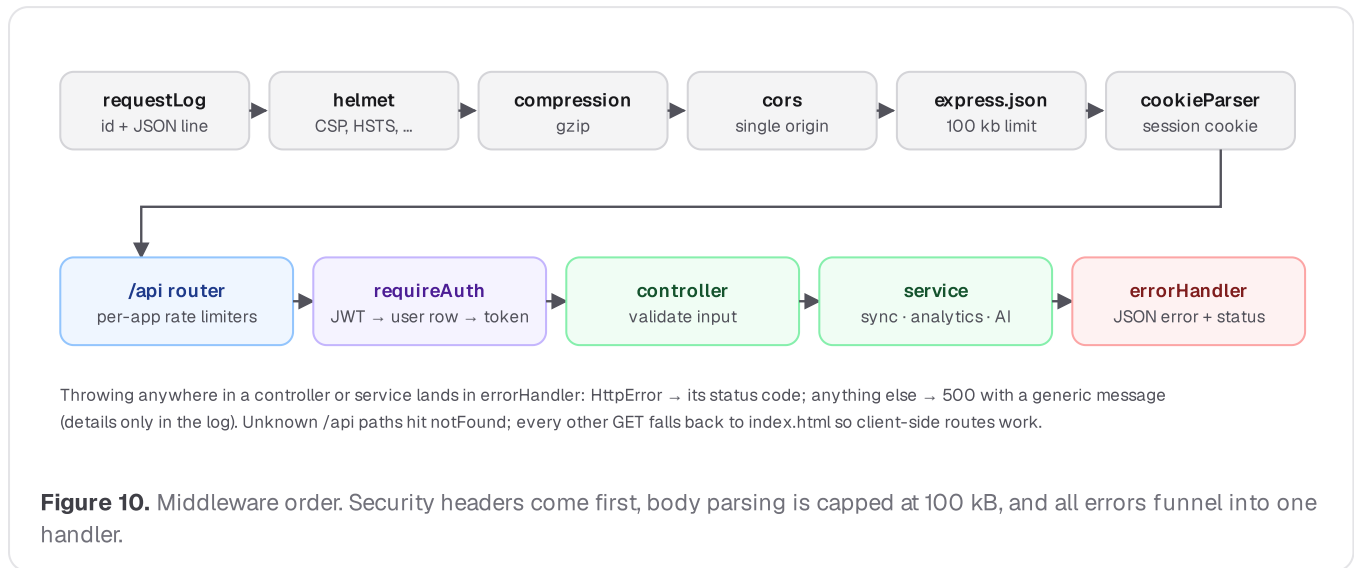
6 Backend deep dive

6.1 Layers and dependency injection



The injection is the most important structural decision. `requireAuth` loads the user row and decrypts the GitHub token; for the shared demo user it substitutes `req.deps = { ...deps, github: demoGithub, config: { ai.provider = 'local' } }`. Controllers always use `req.deps`, so a demo session physically cannot reach GitHub or a paid AI API.

6.2 Request pipeline



6.3 Module reference

File	Responsibility	Notable details
<code>config/env.js</code>	Reads and normalises environment variables	Derives <code>baseUrl</code> from Vercel's production domain; <code>assertProductionConfig</code> refuses to boot without <code>JWT_SECRET</code> , <code>TOKEN_ENCRYPTION_KEY</code> , GitHub client id/secret
<code>config/db.js</code>	pg pool	Pool size 10 (3 on Vercel), 5 s connection timeout, 20 s statement timeout, idle-client error handler so a DB restart cannot crash the process
<code>config/migrate.js</code>	Versioned migrations	Takes <code>pg_advisory_lock</code> , applies each <code>NNN_*.sql</code> once in a transaction, records it in <code>schema_migrations</code>
<code>services/githubService.js</code>	All GitHub I/O	REST (user, repos, commits) + GraphQL (PRs with reviews); <code>fetchWithTimeout</code> ; friendly 401/403/409/451 messages; revoke OAuth grant on account deletion
<code>services/syncService.js</code>	Keeps PostgreSQL in step with GitHub	Incremental sync, in-flight de-duplication, <code>uniqueBy</code> before every bulk upsert, retention pruning
<code>services/analyticsService.js</code>	<code>buildSummary</code> and friends	Pure; also <code>loadRows</code> , <code>applyDataQuality</code>
<code>services/reviewUtil.js</code>	Review semantics	Shared by the real client and the demo generator so both mean the same thing
<code>services/teamService.js</code>	Multi-repository aggregation	<code>mapLimit</code> concurrency 3; at most 5 first-time syncs per request
<code>services/aiService.js</code>	Prompt, provider calls, rule-based report	Gemini (default model <code>gemini-2.5-flash</code>), OpenAI (<code>gpt-4o-mini</code>) or local
<code>services/digestService.js</code>	Schedule, claim, deliver	<code>isDue</code> is pure; <code>runDueDigests</code> claims with one UPDATE

File	Responsibility	Notable details
<code>services/digestRender.js</code>	E-mail HTML/text and Slack blocks	Pure; escapes per target; only emits https links
<code>services/demoData.js</code>	Deterministic fake GitHub	Same interface as <code>githubService</code> ; seeded by calendar day
<code>middleware/auth.js</code>	Session → user → token	Distinguishes expired session from unreadable stored credentials
<code>middleware/errorHandler.js</code>	Uniform JSON errors	<code>{ error, requestId }</code> ; <code>HttpError</code> carries a safe status/message; anything else is a generic 500 with the stack logged
<code>middleware/requestLog.js</code>	Structured logs	One JSON line per request with an id; query strings are never logged (they could contain OAuth codes)
<code>utils/crypto.js</code>	AES-256-GCM helpers	Key = SHA-256 of the secret; random 12-byte IV; format <code>iv.tag.ciphertext</code> (base64url)
<code>utils/validate.js</code>	Input parsing	Repo names <code>[A-Za-z0-9._-]{1,100}</code> excluding "." and "-"; days 1-90; tzOffset -720..840

6.4 Error handling philosophy

- **Expected failures are `HttpError`s** with a status and a message safe to show: 400 bad input, 401 not signed in, 403 forbidden / GitHub scope, 404 not found, 429 rate limit, 502 upstream trouble.
- **Unexpected failures are logged with the request id and returned as a generic 500.** Stack traces and SQL never leave the server.
- **Upstream calls always have timeouts** (`fetchWithTimeout`), so a stalled GitHub or Slack cannot hold a serverless invocation open.
- **A failing optional feature never costs the core one:** if the AI call fails, the digest still goes out with the rule-based summary; if one repository cannot be read, the team view lists it as "left out" and shows the rest.

7 Authentication, authorization and security

7.1 Sign-in with GitHub OAuth

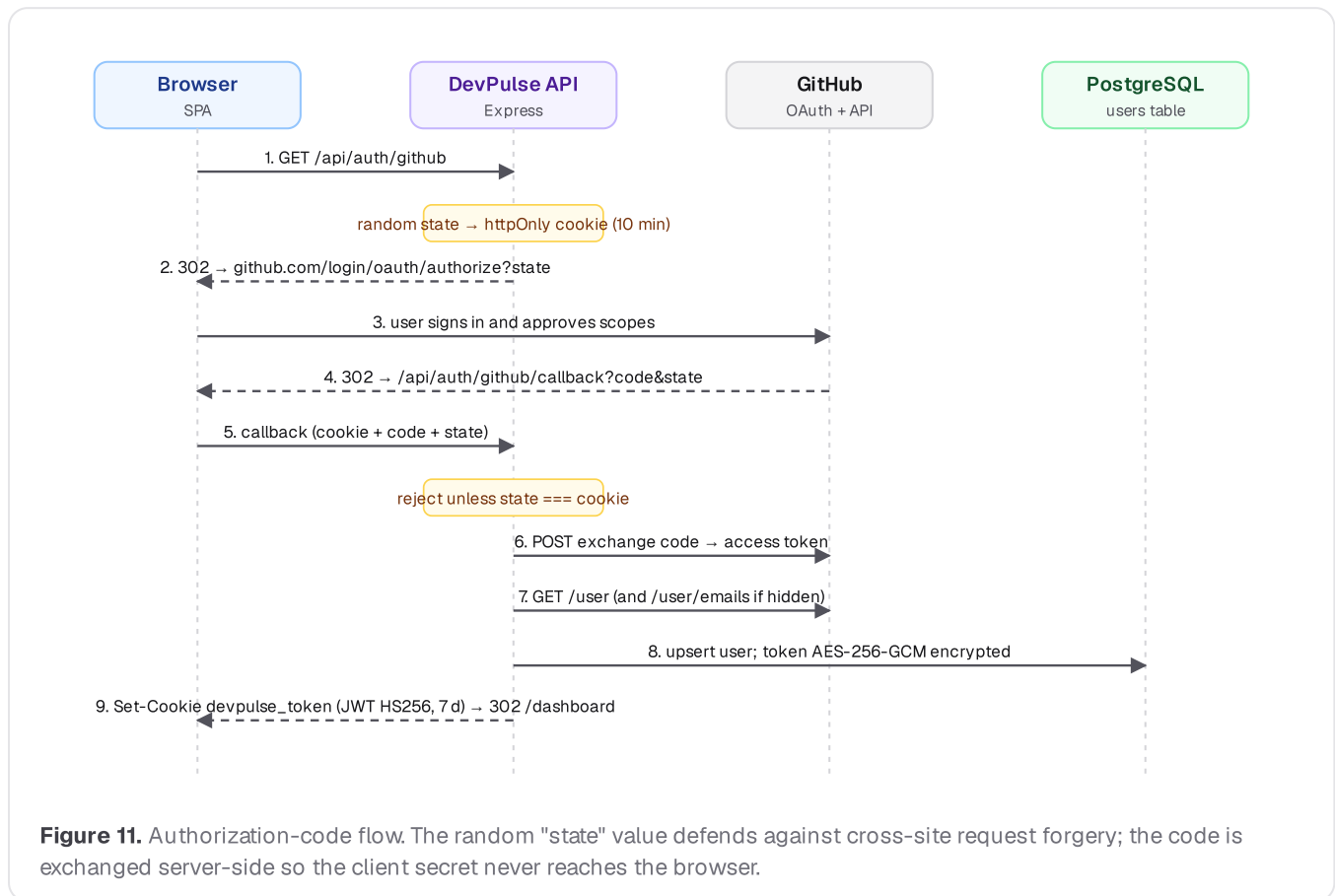


Figure 11. Authorization-code flow. The random "state" value defends against cross-site request forgery; the code is exchanged server-side so the client secret never reaches the browser.

- 1. Start.** GET /api/auth/github generates 16 random bytes (`state`), stores it in a 10-minute `httpOnly`; `SameSite=Lax` cookie (and `Secure` in production) and redirects to GitHub with the client id, callback URL, scopes and the state.
- 2. Consent.** The user approves. Default scope: `read:user user:email repo` (the `repo` scope is required to read *private* repositories; set `GITHUB_SCOPE="read:user user:email public_repo"` to limit DevPulse to public ones).
- 3. Callback.** GitHub redirects with `code` and `state`. The server rejects the request unless `state` equals the cookie, then exchanges the code for an access token and reads `/user` (plus the primary verified e-mail if the profile hides it).
- 4. Persist.** The user row is upserted by `github_id`; the token is stored **AES-256-GCM encrypted**. A session JWT is issued in an `httpOnly` cookie and the browser is redirected to `/dashboard`.

7.2 Sessions

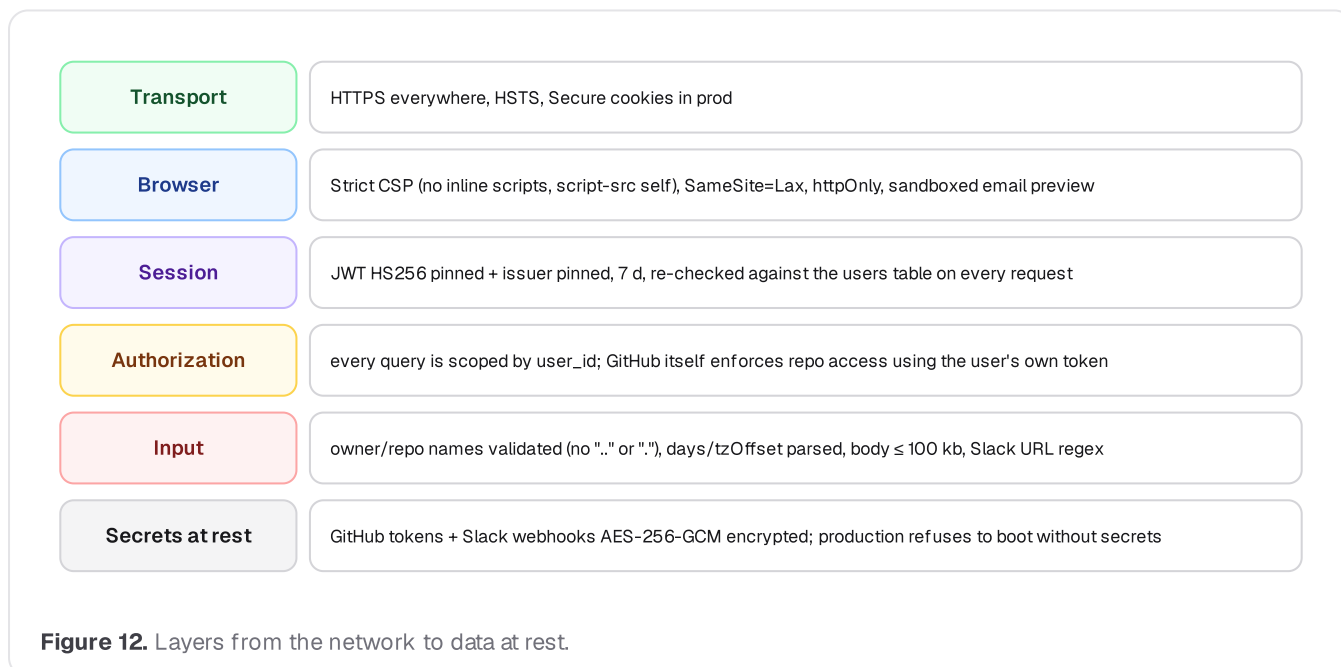
Property	Value	Why
Cookie name	<code>devpulse_token</code>	

Property	Value	Why
Flags	<code>HttpOnly</code> , <code>SameSite=Lax</code> , <code>Secure</code> in production, <code>Path=</code>	JavaScript cannot read it; not sent on cross-site POSTs; HTTPS only in prod
Token	JWT, HS256 pinned , issuer <code>devpulse</code> , payload <code>{sub, login}</code>	Pinning the algorithm on sign <i>and</i> verify rejects forged <code>alg: none</code> tokens
Lifetime	7 days (demo: 24 hours)	Bounded blast radius
Re-validation	Every request re-loads the user row and decrypts the stored GitHub token	Deleting an account or rotating the encryption key takes effect immediately
Expired GitHub token	GitHub 401 → API 401 → SPA shows "session expired" and signs out	The user is told why, not just bounced

7.3 Authorization model

DevPulse does not invent a permission system; it **borrowes GitHub's**. Every GitHub call uses *the signed-in user's own token*, so GitHub itself decides whether that user may read a repository. DevPulse additionally scopes *every* stored row by `user_id`: repositories are unique per `(user_id, full_name)`, and every query joins through the current user. Two accounts that both see the same private repository each hold their own copy, and no endpoint accepts a repository id from the client - only `owner/name`, resolved against the caller's rows.

7.4 Defence in depth



Control	Implementation
Security headers	<code>helmet</code> defaults: HSTS, <code>X-Content-Type-Options</code> , <code>Referrer-Policy: no-referrer</code> , <code>frame-ancestors self</code> , COOP/CORP

Control	Implementation
Content-Security-Policy	<code>script-src 'self'</code> and <code>script-src-attr 'none'</code> : no inline scripts . The theme bootstrap lives in <code>/theme-boot.js</code> for this reason. <code>img-src</code> adds only GitHub avatar hosts.
CORS	Single configured origin with credentials; in production the SPA and API are the same origin
Rate limiting	Per app instance (see table 7.5)
Input validation	Central validators; repository names reject <code>.</code> and <code>..</code> (path traversal into the GitHub URL); <code>days</code> 1-90; <code>tzOffset</code> -720..840; JSON body ≤ 100 kB; Slack URLs must match <code>^https://hooks.slack.com/services/...</code>
SQL injection	Parameterised queries everywhere; bulk upserts pass one JSON parameter to <code>jsonb_to_recordset</code>
Secrets at rest	GitHub tokens and Slack webhooks encrypted with AES-256-GCM (random IV per value, authentication tag verified on decrypt)
Output encoding	React escapes by default; the digest renderer escapes per target (HTML, Slack mrkdown) and only emits <code>https</code> links; the e-mail preview iframe is <code>sandbox=""</code> (no scripts, no forms, no navigation); the Slack preview never uses <code>innerHTML</code>
Logging	Structured JSON with a request id; query strings are never logged (OAuth codes live there)
Fail-fast config	Production refuses to start without <code>JWT_SECRET</code> , <code>TOKEN_ENCRYPTION_KEY</code> , GitHub client id/secret
Right to erasure	<code>DELETE /api/account</code> : revoke the OAuth grant at GitHub, delete the user row, cascade through everything, clear the cookie
Cron endpoint	Does not exist (404) unless <code>CRON_SECRET</code> is set; bearer token compared with <code>timingSafeEqual</code> over SHA-256 digests
Container	Multi-stage Docker image, production dependencies only, runs as the unprivileged <code>node</code> user

7.5 Rate limits

Endpoint(s)	Limit	Window
All authenticated <code>/api</code> routes	600 requests	15 min
Sign-in, callback, demo login	30	15 min
Cron digest (GET/POST)	20	15 min
<code>POST /ai/sprint-summary</code>	20	1 min (plus daily quota for paid providers: <code>AI_DAILY_LIMIT</code> , default 20)
<code>POST /digest/preview</code>	20	1 hour
<code>POST /digest/send-test</code>	5	1 hour
<code>DELETE /account</code>	5	1 hour

Known limitation

Counters live in memory per app instance (so per serverless instance on Vercel). They stop a single noisy client but a determined one could spread requests across instances. Database writes are idempotent, so correctness is unaffected; if abuse mattered, counters would move to Redis (Upstash has a free tier).

7.6 What data is (and is not) collected

Collected	Never collected
Repository metadata (name, description, language, stars, forks, privacy flag)	Source code or diffs
Commit SHA, first line of the message (300 chars max), author login/name, timestamp, URL	Full commit messages, file names, patches
PR number, title, state, draft/bot flags, timestamps, additions/deletions/changed-file counts, author login	PR descriptions, comment bodies, review comment text
Review events: reviewer login, state (approved / changes requested / commented), timestamp, bot flag	Issue contents, secrets, Actions logs, deployment data
Your GitHub profile (login, name, avatar URL, primary verified e-mail)	Anything from repositories you did not open or select

8 Data model and migrations

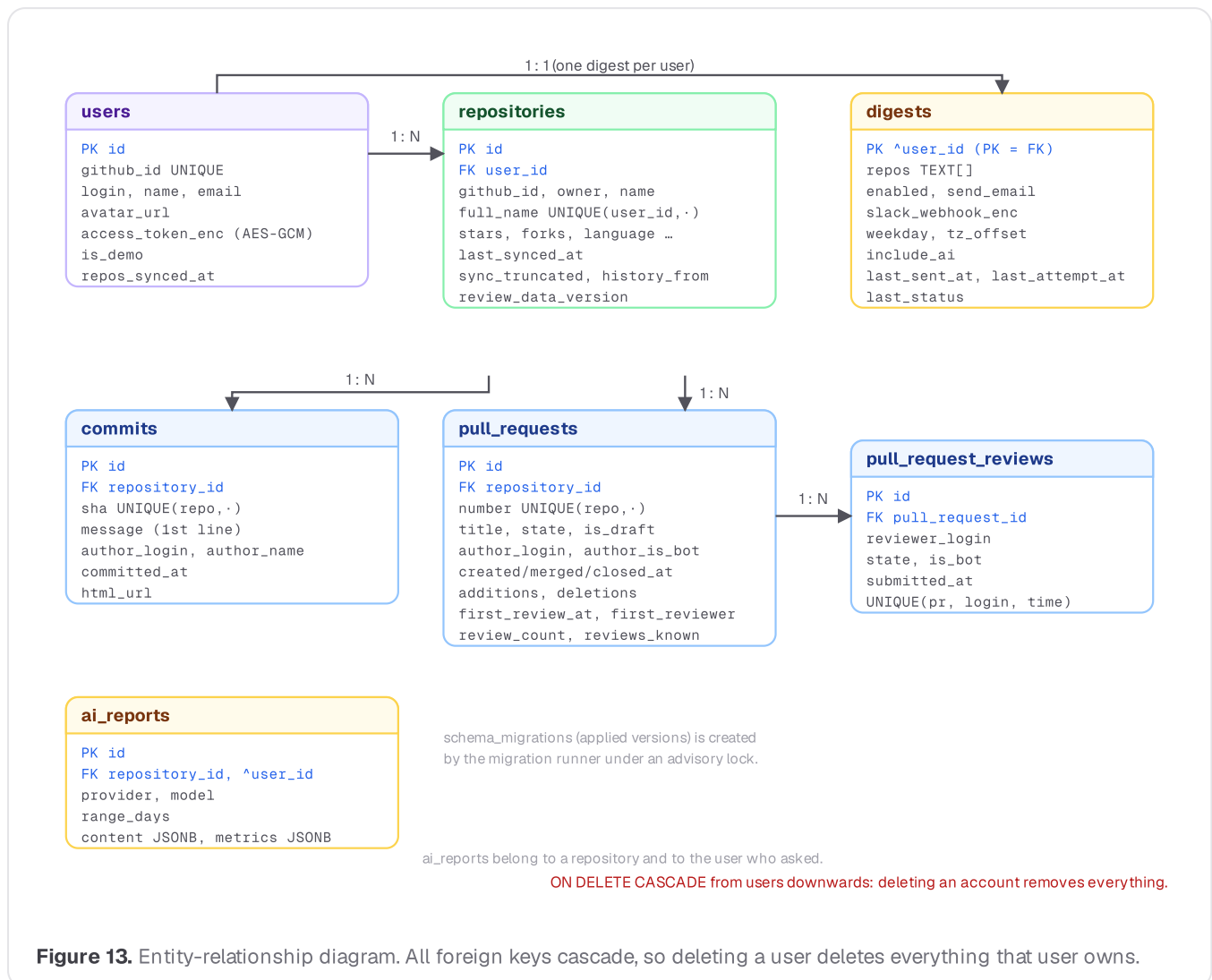


Figure 13. Entity-relationship diagram. All foreign keys cascade, so deleting a user deletes everything that user owns.

8.1 Tables

Table	Purpose	Key constraints / indexes
<code>users</code>	One row per GitHub account plus the shared demo account (<code>github_id = -1</code> , <code>is_demo</code>)	<code>github_id</code> unique; <code>access_token_enc</code> never returned by any endpoint
<code>repositories</code>	A user's view of a repository + sync bookkeeping	<code>UNIQUE(user_id, full_name)</code> ; <code>last_synced_at</code> , <code>sync_truncated</code> , <code>history_from</code> , <code>review_data_version</code>
<code>commits</code>	Commits in the retention window	<code>UNIQUE(repository_id, sha)</code> ; index (<code>repository_id</code> , <code>committed_at</code> DESC)
<code>pull_requests</code>	PRs with size, draft/bot flags and first-review summary	<code>UNIQUE(repository_id, number)</code> ; <code>state</code> CHECK in (open, closed, merged); <code>reviews_known</code> marks rows whose reviews have been fetched

Table	Purpose	Key constraints / indexes
<code>pull_request_reviews</code>	One row per submitted review	<code>UNIQUE(pull_request_id, reviewer_login, submitted_at)</code> ; indexes on PR and time
<code>ai_reports</code>	Generated summaries (JSONB) + the totals they were based on	At most the 20 latest per repository per user are kept
<code>digests</code>	One weekly-digest subscription per user	<code>user_id</code> is both PK and FK; <code>slack_webhook_enc</code> encrypted; partial index on <code>last_sent_at</code> WHERE <code>enabled</code>
<code>schema_migrations</code>	Which migration files have run	Created by the runner

8.2 Migration history

File	Adds	Reason
<code>001_init.sql</code>	users, repositories, commits, pull_requests, ai_reports	Core schema; written to be idempotent so databases created before versioned migrations adopt it cleanly
<code>002_sync_quality_and_demo.sql</code>	<code>sync_truncated</code> , <code>history_from</code> , <code>is_demo</code>	Honest reporting when GitHub pagination cuts history; demo account
<code>003_repo_cache_and_indexes.sql</code>	<code>repos_synced_at</code> , extra indexes	Cache the repository list; speed up window queries
<code>004_reviews_and_size.sql</code>	PR draft/bot/size/first-review columns, <code>pull_request_reviews</code> , <code>review_data_version</code>	Review latency and size analytics; version column triggers a one-time backfill
<code>005_digests.sql</code>	<code>digests</code>	Weekly digest

8.3 Retention and size

Nothing older than the sync window plus a month is ever read, so nothing older is kept: commits older than **210 days** and finished PRs (merged or closed) older than 210 days are deleted at the end of every sync (open PRs are always kept). With the 1,000-commit and 500-PR ceilings, a repository costs on the order of hundreds of kilobytes, which is why the Neon free tier (0.5 GB) is comfortable for personal and small-team use.

8.4 Bulk writes

Every write path sends **one SQL statement per table** using `INSERT ... SELECT ... FROM jsonb_to_recordset($2::jsonb) ... ON CONFLICT ... DO UPDATE`. This avoids thousands of round trips (important with a serverless database) and makes sync idempotent: re-running it can only converge. One subtlety drove a helper: `ON CONFLICT DO UPDATE` cannot touch the same row twice in a statement, and GitHub's paginated lists can repeat an item when activity shifts pages mid-fetch. `uniqueBy()` collapses duplicates (last wins) before every bulk insert.

9 The GitHub sync engine

The sync engine is the only code that talks to GitHub for activity data. Its job: make PostgreSQL contain the last 180 days of activity for a repository, using as few API calls as possible, never leaving a gap, and never doing the same work twice concurrently.

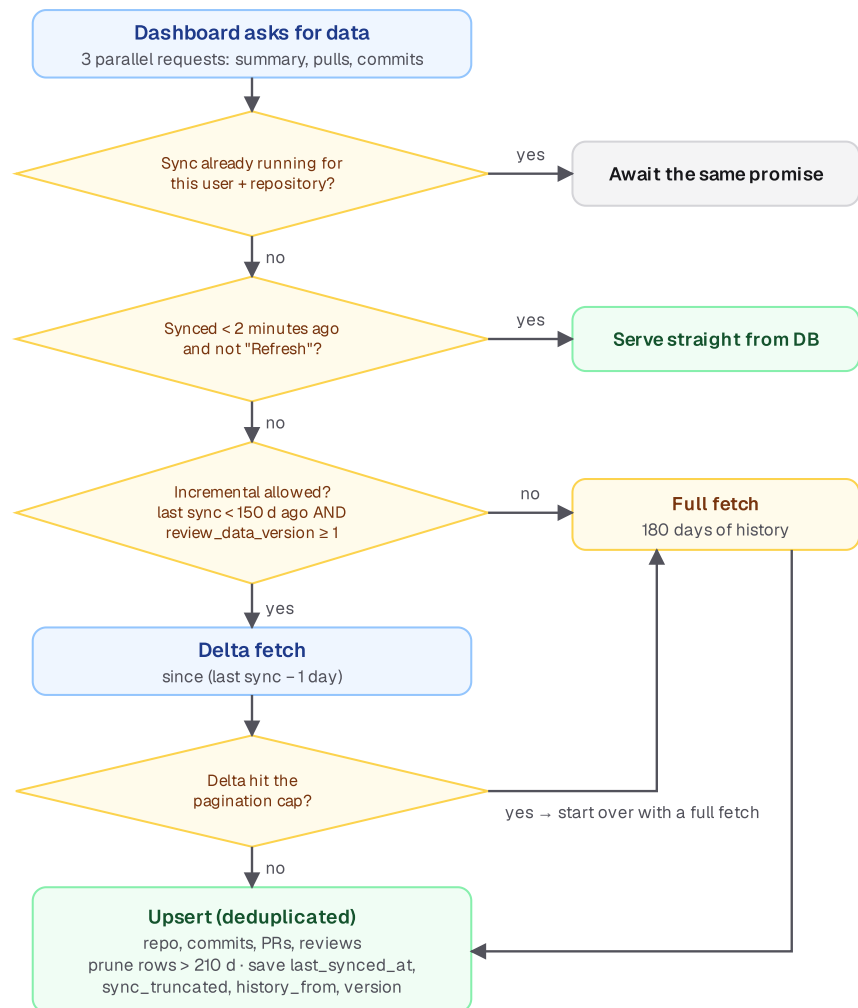


Figure 14. Decision flow of syncRepository(). Four guards (in-flight, freshness, incremental eligibility, delta overflow) keep calls minimal and correct.

9.1 What is fetched

Data	API	Page size / cap	Notes
Repository metadata	REST <code>GET /repos/:owner/:name</code>	1 call	Doubles as the access check for private repositories

Data	API	Page size / cap	Notes
Commits	REST <code>GET /repos/:o/:n/commits?since=</code>	100 per page, 10 pages = 1,000 commits	409 (empty repository) is treated as "no commits"
Pull requests with reviews, size, draft and bot flags	GraphQL <code>pullRequests(orderBy: UPDATED_AT DESC)</code>	50 per page, up to 10 pages = 500 PRs; stops early once <code>updatedAt < since</code>	Each PR carries <code>reviews(first: 30)</code> - the reason GraphQL is used
User, e-mail, repositories list	REST	Paginated	Repository list is cached 5 minutes in the DB

9.2 Incremental sync

The first sync of a repository reads the whole **180-day** window (enough for a 90-day view plus its previous-period comparison). Later syncs ask only for what changed *since the last sync minus one day* (the overlap guards against clock skew and in-flight events). Incremental mode is used only if the last sync was within **150 days** (window minus 30) and `review_data_version ≥ 1`. Otherwise it falls back to a full read. If the delta itself hits a pagination cap, the code abandons it and does a full read rather than leave a hole.

9.3 One sync at a time

The dashboard needs three endpoints at once (summary, pulls, commits). Without protection, three parallel requests would each pull the same repository. A per-`user:owner/repo` `Map` of promises makes concurrent callers share one in-flight sync. The map is per process, so on serverless two cold instances could sync the same repository simultaneously; because all writes are idempotent upserts, the outcome is still correct, merely redundant.

9.4 Freshness cache

If the repository was synced less than **2 minutes** ago, requests are answered from PostgreSQL without contacting GitHub. The **Refresh** button passes `refresh=true` to bypass the cache (and runs a delta sync, not a full one).

9.5 Honest truncation

Because of the 1,000-commit ceiling, a very active repository may have 180 days of PRs but only 40 days of commits. The sync records `sync_truncated` and `history_from` (the oldest commit it actually has). The analytics layer then:

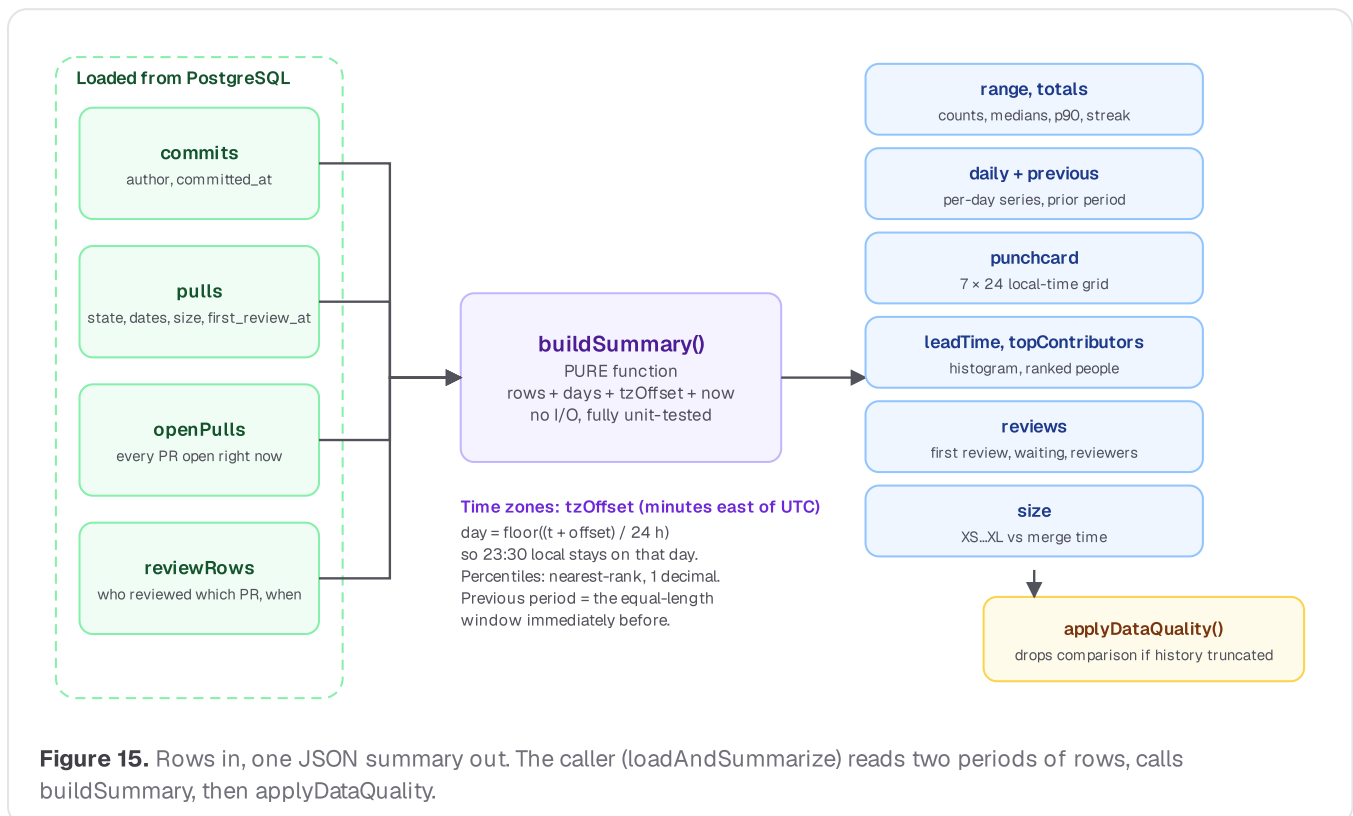
- shows a banner: *"This repository is very busy. DevPulse reads the most recent 1,000 commits, so activity before <date> is incomplete"*;
- **withholds the previous-period comparison** when history is cut inside it, because comparing a full current period with half a previous one would print nonsense like "+315%";
- leaves the current-period numbers visible but labelled.

9.6 Review backfill

Review data arrived in migration 004. Repositories synced before it have `review_data_version = 0`; their next sync is forced to be a *full* one (so old PRs gain review data) and then sets the version to 1. Until a PR's reviews have been fetched, `reviews_known = false` and it is excluded from review metrics, so "unknown" is never confused with "unreviewed". When no PR in the range is known yet, the Reviews page says *"Collecting review data"*.

10 The analytics engine

All numbers on the Overview, Reviews, Pull requests and Insights screens come from one pure function: `buildSummary({ commits, pulls, openPulls, reviewRows, days, now, tzOffset, previousAvailable })`. It performs no I/O and takes the clock as a parameter, which makes it deterministic and easy to test (the analytics suite builds fixtures and asserts exact outputs).



10.1 Time handling

The browser sends `tzoffset` = minutes *east* of UTC (IST = 330). Every timestamp is shifted by the offset *before* it is bucketed: `dayIndex = floor((t + offset) / 86 400 000 ms)`. Consequences: a commit at 23:30 local belongs to the day the author remembers; the punchcard's "Tuesday 14:00" is the viewer's Tuesday 14:00; the "today" bucket is the viewer's today. No SQL date arithmetic is involved, which removes a classic class of off-by-one bugs.

10.2 Windows and the previous period

- **Current period:** the last *N* local days ending today (*N* in 7, 14, 30, 90; any 1-90 is accepted).
- **Previous period:** the equal-length window immediately before it. Used for deltas, the dashed overlay on the activity chart, and review-speed trend.
- Rows are loaded for `2N + 2` days: two periods plus a day of slack on either side so time-zone shifts never clip a boundary day.

- `previousAvailable` is true only if `2N ≤ 180` (the sync window) *and* data quality allows it.

10.3 Metric definitions

Metric	Definition
Commits	Commits whose local day falls in the window
Pull requests (opened)	PRs created in the window; split by current state (merged / open / closed)
Merged	PRs whose <code>merged_at</code> falls in the window (regardless of when opened)
Time to merge (lead time)	<code>merged_at - created_at</code> for PRs merged in the window; median, p90, mean and a 6-bucket histogram (<1h, 1-4h, 4-24h, 1-3d, 3-7d, >7d)
Merge rate	<code>merged / (merged + closed without merge)</code> for PRs opened in the window
Contributors	Distinct people (login, else name) with a commit or an opened PR; ranked by commits + PRs, top 8 returned
Active days / longest streak	Days with ≥ 1 commit; longest run of consecutive such days
Stale PRs	Open PRs older than 14 days ; also "oldest open (days)"
Busiest day	Day with the most commits
Punchcard	7 × 24 matrix of commit counts by local weekday and hour
Percentiles	Nearest-rank : index = <code>ceil(p × n) - 1</code> on the sorted list, rounded to one decimal. Chosen over interpolation because it always returns an observed value

10.4 Health signals (no AI involved)

Signals translate numbers into a rating (good / warn / bad / neutral) and one sentence. They are computed in the browser from the summary (`lib/signals.js`), so they are the same everywhere they appear (Insights, Team, landing demo).

Signal	Good	Watch	Bad	Sentence says
Review speed (median time to merge)	≤ 24 h	≤ 72 h	> 72 h	Median open-to-merge, and the slowest 10 %
First review	≤ 8 h	≤ 24 h	> 24 h	Median wait, % reviewed within a day, how many still waiting
Review load (needs ≥ 2 reviewers)	top ≤ 50 %	≤ 70 %	> 70 %	Who did what share of reviews
Stale pull requests	0	1-2	≥ 3	How old the oldest open PR is
Knowledge spread	top author ≤ 50 %	≤ 75 %	> 75 %	Share of commits by the top author
Work rhythm	off-hours < 15 %	< 30 %	≥ 30 %	Share of commits on weekends or 22:00-06:00 local
Momentum	≥ -10 %	≥ -35 %	< -35 %	Commits vs. previous period (only when comparable)

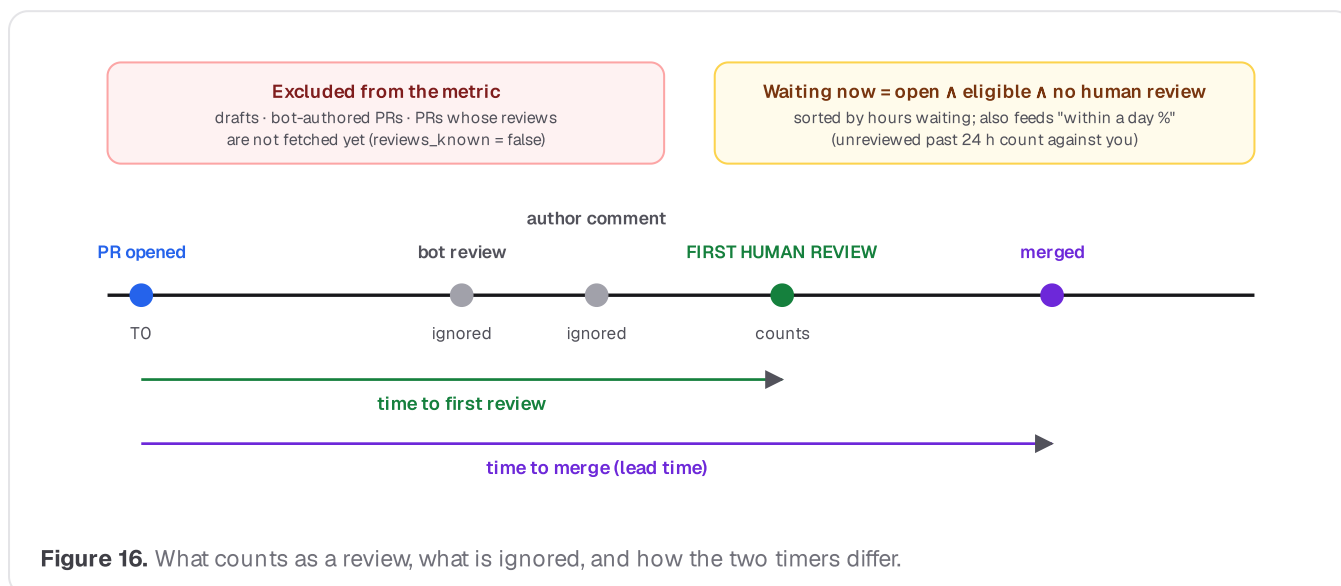
Thresholds are intentionally simple and visible so they can be argued with. They are heuristics for conversation, not performance targets.

10.5 Data-quality pass

`applyDataQuality(summary, repo, days)` reads the repository's `sync_truncated / history_from`. If history was cut inside the previous period it sets `previous = null`, `previousCommitsDaily = null`, `range.previousAvailable = false` and attaches `dataQuality { incomplete, historyFrom }`, which the UI turns into the amber banner.

11 Review analytics

This is the feature set GitHub's repository Insights do not provide. Precision of definition matters more than anything else here, because "time to review" is easy to measure in a misleading way.



11.1 What is a review?

A **review** is a *submitted* pull-request review (approved, changes requested, or commented) by someone **other than the author**. Plain issue-style comments are not reviews. Reviews in the `PENDING` state are ignored (they are drafts only the reviewer can see). Bot reviews (dependabot, Copilot, CI) are kept as events but are flagged and **never** count as the first human review nor towards reviewer load. This semantic lives in one file (`reviewUtil.js`), shared by the real GitHub client and the demo generator so both mean exactly the same thing.

11.2 Eligibility

A PR enters the review metrics only if it is **not a draft**, **not authored by a bot** and **its reviews are known** (`reviews_known`). Drafts are excluded because nobody is expected to review them; bot PRs because they would drown the signal (dependency bumps auto-merge); unknown PRs so that missing data is never reported as "no review".

11.3 The numbers

Number	How it is computed
Time to first review	For eligible PRs opened in the window: <code>first_review_at - created_at</code> (floored at 0). Median, p90 and a histogram on the same buckets as lead time.
Reviewed within a day %	(reviewed within 24 h) ÷ (reviewed + still-unreviewed PRs that are already older than 24 h). Unreviewed-but-old PRs <i>count against</i> you; a PR opened an hour ago does not.

Number	How it is computed
Waiting now	Open, eligible, no human review yet - regardless of when opened. Sorted by hours waiting; top 10 returned with the true total.
Reviewer load	Submitted human reviews inside the window by reviewer: count, share, approvals, changes requested, PRs reviewed, median response time (their first touch per PR minus PR creation).
Top reviewer share	Reviews by the busiest reviewer ÷ all reviews. Becomes a signal only with ≥ 2 reviewers.
Trend	Previous-period median first review (when comparable) feeds the delta chip.

11.4 Size versus speed

Size = `additions + deletions`. Buckets: **XS** < 10 lines, **S** 10-49, **M** 50-249, **L** 250-999, **XL** 1,000+. For PRs merged in the window the median time to merge is computed per bucket. `sizeInsight()` writes a sentence only when at least two buckets have ≥ 2 merged PRs: either "*Large pull requests merge $N\times$ slower than small ones*" or "*Merge time barely changes with size here*". It never claims causation.

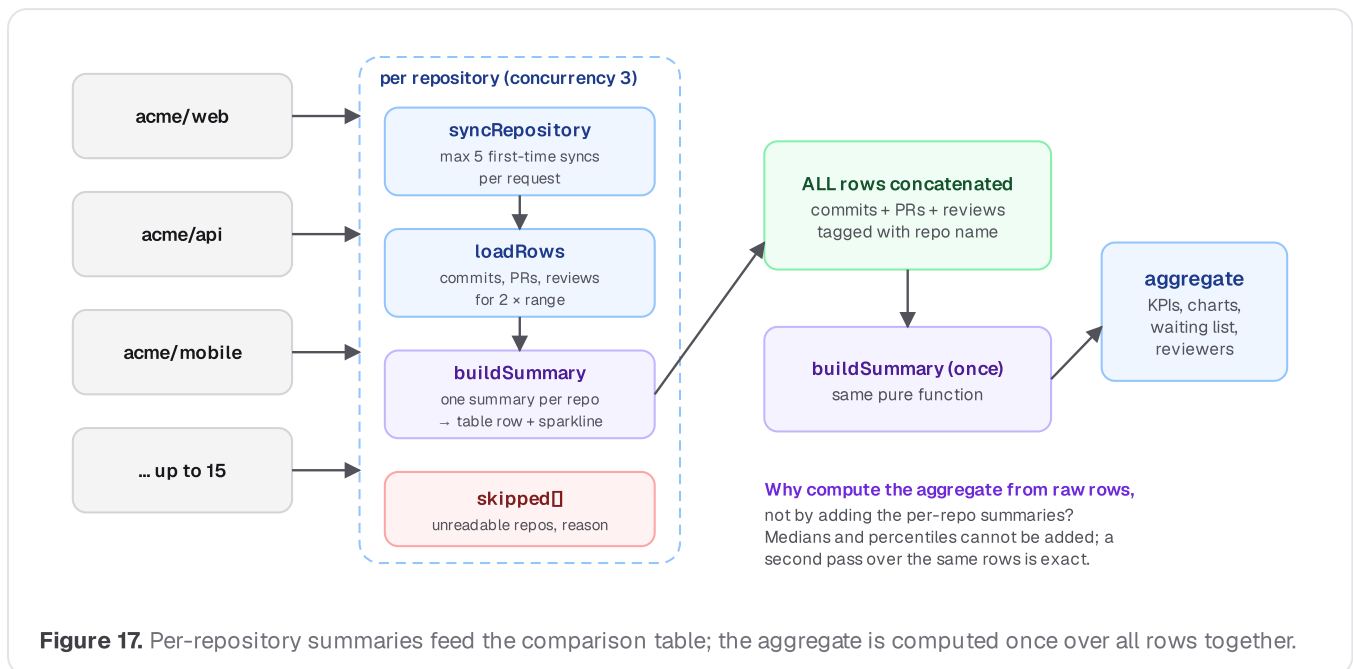
11.5 Why GraphQL

The REST API would require one extra request per PR to list reviews - hundreds of calls for a busy repository, and a burned rate limit on a serverless function. GraphQL returns 50 PRs with their first 30 reviews, size and author type in one round trip, which is what makes the Reviews page affordable on a free tier.

Limits of the review data

Only the first 30 reviews per PR are read. The 500-most-recently-updated-PR ceiling applies. Reviews requested but not yet submitted (review requests) are not visible, so a PR waiting on a *specific* person looks the same as one nobody was asked about.

12 Team view (multi-repository)



12.1 Request

GET /api/team/summary?repos=a/b,c/d&days=30&tzOffset=330&refresh=true. Without `repos` the server uses the user's analyzed repositories (most recently pushed first, max 15). More than 15 names is rejected with 400.

12.2 Processing

1. Look up which requested repositories have never been synced.
2. Run `syncRepository` + `loadRows` for each with **concurrency 3** (`mapLimit`). Only **5 first-time syncs** are allowed per request so the call stays inside a serverless time budget; the rest are reported in `skipped` with "open it once first".
3. A repository that fails (404, 403, 451, GitHub rate limit) is reported in `skipped` with GitHub's reason; a 401 aborts the request (the session itself is bad).
4. Build a per-repository summary (for the table) **and** concatenate all rows (tagging open PRs with their repository) into one `buildSummary` call for the aggregate.
5. Previous-period comparison for the aggregate requires *every* repository to have a comparable previous period.

12.3 Why compute the aggregate from raw rows?

Medians, percentiles and "distinct contributors" cannot be obtained by adding or averaging per-repository results (the median of medians is not the median). Re-running the same pure function over the union of rows is exact and guarantees that team totals equal the sum of the parts - a property the test-suite asserts (`team.test.js`).

12.4 Output

`{ range, aggregate, repos[], skipped[] }`. `repos[]` rows carry commits, previous commits, PRs merged/open/stale, median merge hours, median first-review hours, waiting count, a daily series for the sparkline and an `incomplete` flag. `aggregate.dataQuality.incompleteRepos` lists busy repositories whose history is cut.

13 AI summaries

13.1 What is sent to the model

Never source code, commit messages or PR titles. `buildPrompt()` assembles one JSON object of **aggregate numbers**: period length, totals (commits, PRs by state, contributors), median/p90 merge hours, merge rate, stale count and oldest open PR age, first-review median, % reviewed within 24 h, count waiting, top-reviewer share, median PR size, active days, longest streak, the previous period's totals, the busiest day, up to **five** top contributors as login + counts, and the commits-per-day array. The model is asked to answer *only* with JSON `{summary, insights[], suggestions[]}` and not to invent data.

13.2 Providers and fallbacks

Provider	Selected when	Call
Gemini	<code>AI_PROVIDER=gemini</code> or only <code>GEMINI_API_KEY</code> set	<code>generateContent</code> with <code>responseMimeType: application/json</code> , temperature 0.4, 30 s timeout; default model <code>gemini-2.5-flash</code>
OpenAI	<code>AI_PROVIDER=openai</code> or only <code>OPENAI_API_KEY</code> set	Chat Completions with <code>response_format: json_object</code> , temperature 0.4; default <code>gpt-4o-mini</code>
Local (rule-based)	No key, demo user, previews	Deterministic sentences built from the same metrics; always available and free

`parseReport()` tolerates code fences, validates shape, caps each list at 8 items and converts everything to strings, so a model that misbehaves yields a clear 502 ("unreadable response") rather than corrupt UI.

13.3 Cost control

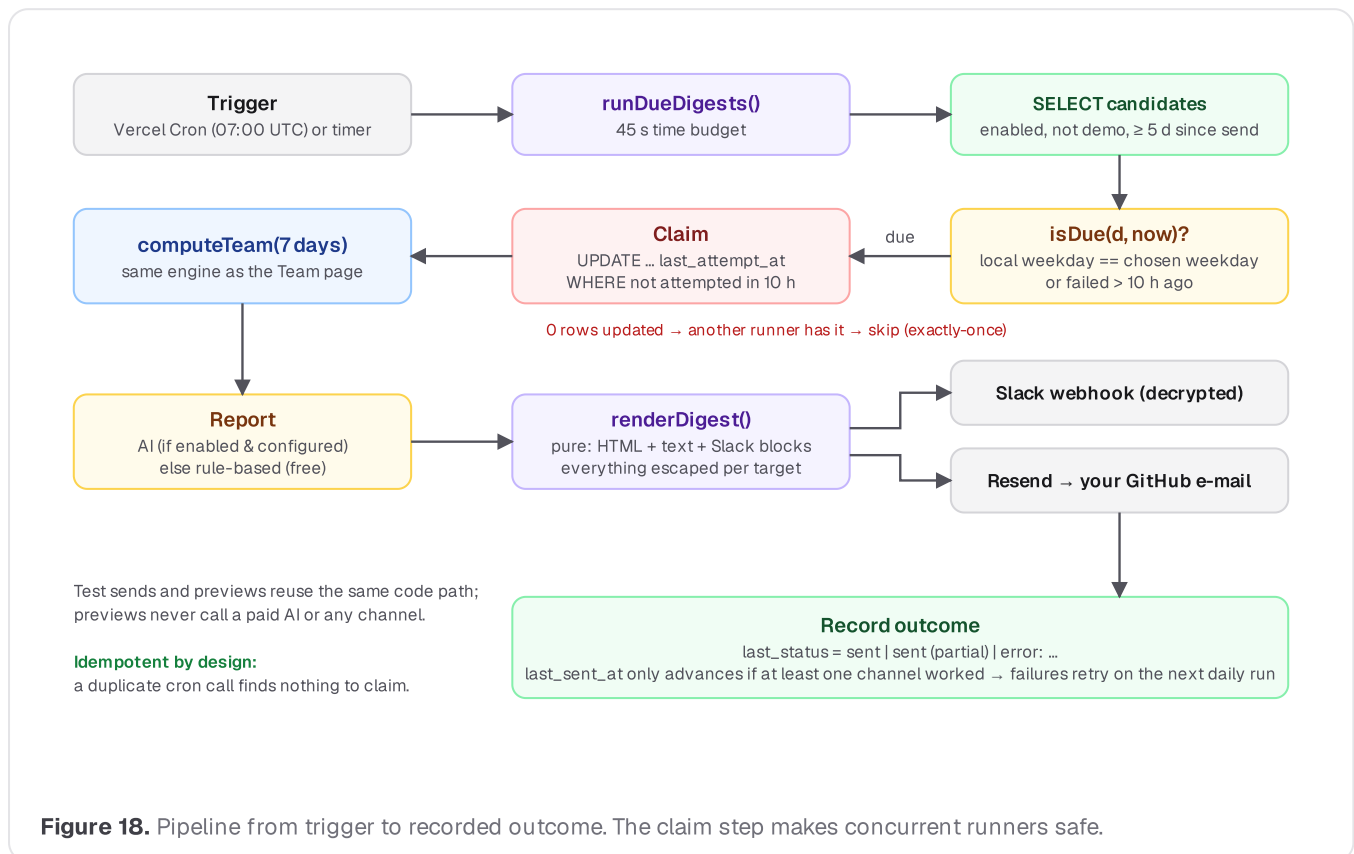
- Paid providers are limited per user to **20 reports per rolling 24 h** (`AI_DAILY_LIMIT`, 0 = unlimited); rule-based ones are not counted.
- The demo user is forced onto `local` by `requireAuth`, so a public demo cannot spend a key.
- Reports are stored with the totals they were based on; only the latest 20 per repository per user are kept.
- The digest uses the AI only for *real* sends with `include_ai` on; previews never do, and an AI failure silently falls back to the rule-based text.

13.4 What the rule-based report says

Example insights: commit and PR counts, delta vs. previous period, busiest day, who led, median merge time, median first-review time. Example suggestions (each conditional): N PRs waiting for a first review (longest wait), one reviewer did ≥ 60 % of reviews, N stale PRs, median turnaround above 48 h, single contributor, sharp drop, no commits. It is deliberately modest: it restates facts and flags thresholds without speculation.

14 Weekly digest

The digest turns DevPulse from a place you visit into a message you receive. It is a small distributed-systems problem: it must send exactly once per week per user, survive crashes and retries, never email the wrong person, and never leak anything.



14.1 Scheduling

Host	Trigger	Notes
Vercel	Vercel Cron: <code>0 7 * * *</code> → <code>GET /api/cron/digest</code> with <code>Authorization: Bearer \$CRON_SECRET</code>	Hobby plans allow daily crons; the endpoint is 404 without <code>CRON_SECRET</code>
Long-running server	In-process timer wakes every 30 min, acts only during <code>DIGEST_HOUR_UTC</code> (default 7), guarded by a Postgres advisory lock that is always released in <code>finally</code>	Disable with <code>DIGEST_SCHEDULER=off</code>

The runner is the same either way: `runDueDigests()` loads up to 200 enabled, non-demo subscriptions not sent in the last 5 days, and processes them within a **45-second budget** (the remainder are "deferred" to the next run).

14.2 Who is due?

`isDue(d, now)` is pure: the subscription is enabled; it was not sent within **5 days**; and the user's *local* weekday (UTC time shifted by the stored `tz_offset`) equals the chosen weekday - *or* the previous attempt ended in an error more than **10 hours** ago (retry on the next daily run rather than waiting a week).

14.3 Exactly-once claim

Before doing any work the runner executes a single statement: `UPDATE digests SET last_attempt_at = now() WHERE user_id = $1 AND (last_attempt_at IS NULL OR last_attempt_at < now() - 10 h) RETURNING user_id`. Only one concurrent runner gets a row back; the others see 0 rows and skip. No separate lock table or distributed lock is needed. `last_sent_at` advances only if at least one channel succeeded, so a total failure is retried but a success is never repeated.

14.4 Content

The model is built from `computeTeam` over the last 7 days (up to **10** repositories): four stats with change vs. the previous week (commits, PRs merged, median time to merge, median time to first review), the top 5 PRs waiting for a first review with their wait, stale count, top reviewers and contributors, the five most active repositories with deep links, an optional paragraph + up to three suggestions, a note about incomplete history, and links to the team view and to the settings page ("Change or stop it").

14.5 Rendering and safety

- **HTML e-mail**: table-based, inline-styled for e-mail clients; every dynamic value passes `escHtml`; links must match `^https://...` or they are dropped.
- **Plain text**: same data, no markup, for clients that prefer it.
- **Slack Block Kit**: header, context, stat fields, waiting list with `<url|label>` links, summary, button; text escaped with Slack's three-character rule (`& < >`); lists clipped to Slack's limits.
- **Preview** in the UI renders exactly these outputs: the e-mail in a sandboxed iframe, Slack through a React renderer that never touches `innerHTML`.

14.6 Delivery

Channel	Mechanism	Security
Slack	POST the blocks to an incoming-webhook URL the user pasted	URL must match <code>https://hooks.slack.com/services/...</code> ; stored encrypted (it is a credential); never returned by the API - only a "connected" boolean
E-mail	Resend HTTPS API (<code>RESEND_API_KEY</code> , <code>DIGEST_FROM</code>) to the user's verified GitHub primary e-mail	Only available if the server is configured and the user has an address; address shown masked (<code>ka****@example.com</code>)

Outcomes are summarised in `last_status`: `sent`, `sent (partial): ...` or `error: ...`, and surfaced as a badge on the Digest page.

15 The demo workspace

"Explore the live demo" signs the visitor into a shared account named `demo` with generated data. It exists so anyone - a reviewer, a classmate, a curious engineer - can evaluate the product in one click, with no GitHub account, no keys and no risk.

15.1 It is not a mock-up

The demo implements **the same interface as** `githubService` (`listRepositories`, `getRepository`, `listCommits`, `listPullRequests` ...). `requireAuth` swaps it in for the demo user, so the demo goes through the real sync → PostgreSQL → analytics → API → React path. Bugs in the real pipeline therefore show up in the demo (and in the end-to-end tests that run against it).

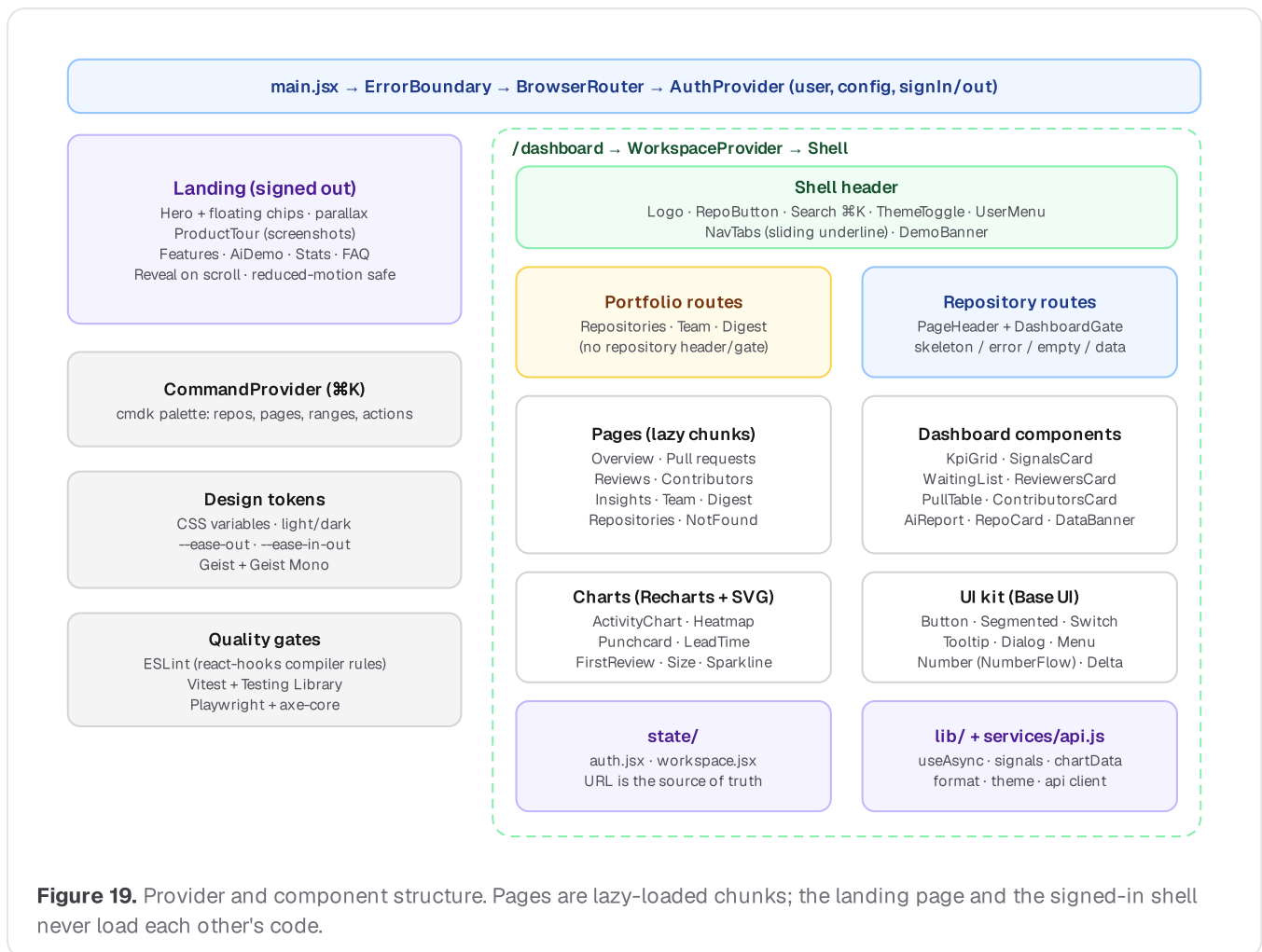
15.2 What it generates

- Nine repositories under `acme/`: **web** (busy, healthy), **api**, **mobile** (slow reviews), **design-system** (one maintainer), a repository with a very long name (to test layout), **monorepo** (so busy that it hits the 1,000-commit cap and shows the incomplete-history banner), **legacy-monolith** (went quiet), **hello-world** (single contributor) and **empty**.
- Seven named people with different weights, lead times and one "night owl"; one commit author with no linked GitHub account; `dependabot[bot]` PRs.
- Pull requests whose lead time correlates with their size, reviews concentrated on a few people, some PRs deliberately left unreviewed and stale, drafts and bot PRs - so every analytics branch has something to show.
- Determinism: data is seeded by repository name and calendar day, and all dates are relative to "now", so the demo never looks stale yet is stable within a day.

15.3 Guard rails

- Cannot be deleted (`DELETE /account` → 403) or send digests (preview only).
- Forced onto the rule-based AI provider; cannot spend a key.
- Session lasts 24 hours; the endpoint is rate limited (30 / 15 min) and can be disabled with `DEMO_ENABLED=false`.
- A banner on every screen says the data is generated, with an "Exit demo" link.

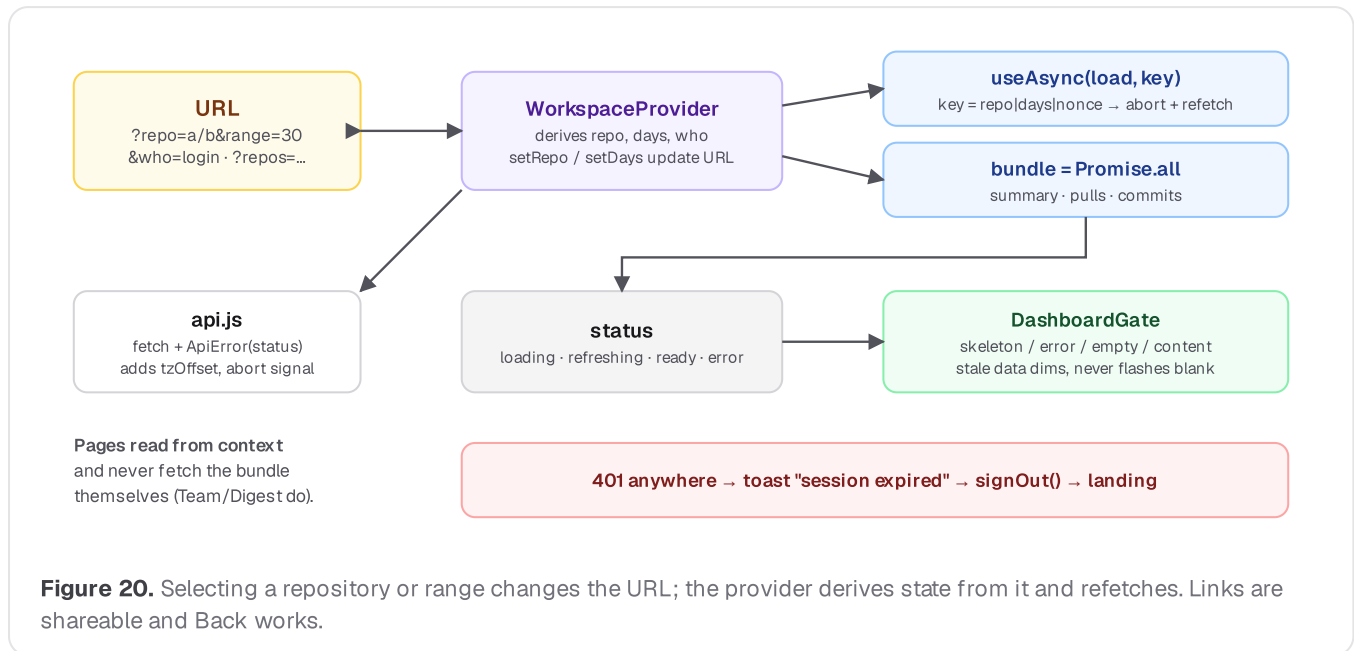
16 Frontend architecture



16.1 Routing and code splitting

React Router 7. `/` shows the landing page (or redirects to `/dashboard` when signed in). `/dashboard` renders the lazily-loaded `Shell` inside a `WorkspaceProvider`, with children `index` (Overview), `repositories`, `team`, `digest`, `reviews`, `pulls`, `people`, `insights`; everything else is a 404 page. Charts and the command menu live in the post-sign-in chunk, so the landing page stays light. **Portfolio routes** (repositories, team, digest) skip the per-repository header and gate; **repository routes** render `PageHeader` and `DashboardGate`.

16.2 State: the URL is the source of truth



URL parameter	Meaning	Owner
repo=owner/name	Selected repository (defaults to last used, remembered in <code>localStorage</code> , else the first)	WorkspaceProvider
range=7 14 30 90	Time range in days (default 30)	WorkspaceProvider
who=login	Contributor filter (ignored if that person is not in the data)	WorkspaceProvider
repos=a/b,c/d	Repositories in the Team view / prefill for the digest	Team page

16.3 Data fetching: `useAsync`

A 20-line hook: `useAsync(load, key)` runs `load(signal)` whenever `key` changes and **aborts the previous run**. "Loading" is *derived* (the stored result belongs to a different key), not set inside an effect - so there is no cascading re-render (and the React compiler lint rules are satisfied). The previous value stays available while the next loads, which is how the dashboard **dims stale numbers instead of flashing a skeleton** on every range change. `key = null` stays idle.

16.4 Statuses

Status	Shown as
loading (no data yet)	Skeleton shaped like the final page (<code>aria-busy="true"</code>)
refreshing (data exists, new request in flight)	Existing content dims slightly (<code>.swap[data-pending]</code>), no layout jump
error with no data	Error card with the server's message and "Try again"; 404 gets "Repository not found"
error with data	Inline red alert above the (still visible) stale content

Status	Shown as
empty	Designed empty states with an explanation and sometimes an action
401	Toast "Your session expired", sign out, back to landing

16.5 Key components

Component	Notes
<code>ActivityChart</code>	Recharts area chart; metric switch, daily/weekly, previous-period overlay, contributor filter chip, per-day top-3 contributors in the tooltip, text alternative with total and peak
<code>Heatmap</code>	Hand-built SVG-free grid (CSS) of weeks × 7 days; keyboard-focusable scroll region on small screens; custom tooltip
<code>Punchcard</code>	7 × 24 bubbles in local time; tooltip with weekday/hour; legend
<code>Sparkline</code>	Monotone cubic (Fritsch-Carlson) path that never overshoots; hover scrubs the KPI value; optional draw-in animation on first paint
<code>KpiGrid</code>	Four cards with <code>NumberFlow</code> digits, Delta chips, sparkline or histogram; third card falls back to Contributors while review data is being collected
<code>SignalsCard</code>	Colour-coded list from <code>deriveSignals</code>
<code>WaitingList</code> / <code>ReviewersCard</code> / <code>FirstReviewChart</code> / <code>SizeChart</code>	The review analytics UI; waiting items tinted amber after 24 h and red after 72 h
<code>RepoCompare</code>	Accessible sortable table (<code>aria-sort</code> , buttons in headers); missing values always sort last
<code>RepoPicker</code>	Dialog with filter and checkboxes; max configurable (15 for Team, 10 for Digest, matching the server)
<code>SlackPreview</code>	Parses Slack mrkdwn into React nodes - no <code>innerHTML</code>
<code>CommandMenu</code>	cmdk inside a Base UI dialog; repositories, pages, ranges, actions
<code>NavTabs</code>	Sliding underline moved with <code>transform</code> only; re-measured when fonts load

16.6 Libraries

Library	Used for	Why this one
React 19 + React Compiler lint rules	UI	Concurrent rendering, effect events (<code>useEffectEvent</code>) used in <code>useAsync</code>
Vite 8	Build / dev server	Fast, route-level code splitting
Tailwind CSS v4	Styling	Tokens as CSS variables; <code>@theme inline</code>
Base UI	Tabs, Dialog, AlertDialog, Menu, Tooltip, Switch, Accordion	Unstyled, accessible primitives with focus management and animation attributes
cmdk	Command palette	Keyboard-first search list

Library	Used for	Why this one
Recharts	Main charts	Declarative, accessible-enough, themable via CSS variables
@number-flow/react	Animated digits	Smooth numeric transitions; wrapped to also expose plain text to screen readers
Sonner	Toasts	Small, accessible
lucide-react	Icons	Consistent set (no brand icons - a custom GitHub icon is included)
Geist / Geist Mono	Type	Self-hosted variable fonts via Fontsource (no third-party font request)

17 Design system and motion

The brief was "production-grade, like Vercel, with the best possible animation", using Emil Kowalski's design-engineering rules. Those rules were applied as concrete, checkable constraints rather than taste.

17.1 Visual language

- **Tokens, not values.** Colours (`--bg`, `--surface`, `--fg`, `--ring`, `--accent`, status hues and their soft backgrounds), radii, shadows and easing curves are CSS variables. Dark mode is a variable swap under `[data-theme]`, guarded for the system preference.
- **Hairlines over borders.** Surfaces are separated by 1 px ring shadows; cards use `box-shadow` rings, which animate cheaply and do not shift layout.
- **Typography.** Geist for UI, Geist Mono for code/shas; tabular numerals (`.num`) so columns of figures align; tight tracking on headings.
- **Density.** 13 px body in dashboards, 28 px KPI numerals, 8-point-ish spacing, max content width 1,200 px, responsive down to 320 px.
- **Colour is never the only signal.** Status uses an icon, a word or a label as well as hue; contrast is verified by axe in both themes.

17.2 Motion rules and how they were applied

Rule	Implementation
Use strong custom curves, not the browser defaults	<code>--ease-out: cubic-bezier(0.23, 1, 0.32, 1)</code> for things entering/responding; <code>--ease-in-out</code> for things already on screen that move (tab underline, segmented control thumb)
Animate only <code>transform</code> and <code>opacity</code>	Entrances (<code>.rise</code>), the tab underline (translate + scaleX), toggles (translate), popovers/modals (scale + opacity); no layout-property animation
Everything pressable feels pressed	<code>.press</code> utility: <code>scale(0.97)</code> on <code>:active</code> with a short transition
Popovers grow from their trigger	Origin-aware transform-origin on menus/tooltips; modals scale from 0.96 and fade
Keyboard-initiated actions do not animate	Command menu opened with Ctrl/Cmd+K appears instantly (<code>data-instant-open</code>); opened by click it eases in
Entrance animation plays once per session	<code>useFirstOnly(key)</code> remembers which blocks already animated, so switching tabs or ranges never replays them
Content swaps are masked	Stale data dims with a short blur/opacity transition; skeletons mirror final layout so nothing jumps
Marketing may be expressive, product must be quiet	Landing: parallax chips, floating idle loops, scroll reveals, auto-scrolling screenshots, word-by-word AI text. Dashboard: short, functional transitions
Reduced motion means gentler, not none	Under <code>prefers-reduced-motion</code> movement is removed but opacity fades and state changes remain; tests emulate the media query to prove the aurora, floats and reveals stop

17.3 Accessibility

- axe-core WCAG 2.1 A/AA scan of the landing page, every dashboard page, the command menu and a phone-width landing page, in **light and dark**, on every CI run.
- Animated numbers render their value as screen-reader text and mark the digit reels `aria-hidden`.
- Charts are `role="img"` with a generated description; the heatmap and tables expose rows/cells with labels.
- Scroll regions that overflow are keyboard-focusable; dialogs trap focus (Base UI); focus rings are visible; all controls have accessible names.
- The e-mail preview iframe has a title; the page uses a real `h1` per route and updates `document.title`.

17.4 Landing page anatomy

Section	Purpose	Notes
Nav + hero	State the promise; offer demo or GitHub sign-in	Headline typography was redesigned after review to avoid a generic "AI-made" look; floating chips use three nested elements (pointer parallax / entrance / idle float) so transforms never fight
Stats + trust marquee	Social proof of technology and scope	
Product tour	Real screenshots of eight pages, auto-scrolling	Screenshots are generated from the real app by a Playwright script and converted to WebP (<code>npm run landing:assets</code>)
Features	Six feature rows and six smaller cards, led by what GitHub does not show	Interactive sample charts embedded live
AI demo	A sample report that writes itself	Words fade in sequentially once visible
How it works, FAQ, final CTA, footer	Answer objections (privacy, access, limits, deletion)	Accordion built on Base UI

18 Deployment and configuration

18.1 Cost-driven choice

Hosting cost was an explicit requirement. The comparison that drove the default:

Option	Cost profile	Fit
Vercel Hobby + Neon Free	Free: CDN for the SPA, serverless function (up to 300 s with Fluid compute; this project sets 60 s), one daily cron; Neon 0.5 GB storage and ~100 compute-hours per month with scale-to-zero	Default for personal and small-team use
Railway	Usage-based with a \$5/month minimum on the Hobby plan	Fine if you want an always-on container and in-process scheduling
Any Docker host	Whatever the host charges	Image is self-contained; needs a PostgreSQL URL

Free-tier numbers are quoted as verified during development and may change; check the providers' current pricing pages before relying on them.

18.2 Vercel + Neon, step by step

1. **Neon:** create a project; copy the *pooled* connection string (host contains `-pooler`). Use it as `DATABASE_URL`.
2. **GitHub OAuth app** (Settings → Developer settings): Homepage `https://<project>.vercel.app`, Authorization callback `https://<project>.vercel.app/api/auth/github/callback`. Keep the client id; generate a client secret.
3. **Vercel:** import the repository (framework "Other"; `vercel.json` sets install, build, output `frontend/dist`, the function and the cron).
4. **Environment variables** (Production): `DATABASE_URL`, `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`, `JWT_SECRET`, `TOKEN_ENCRYPTION_KEY` (two different values from `openssl rand -hex 32`); optional AI, digest and domain variables below.
5. **Deploy**, then check `/api/health` → `{"status": "ok"}`, the demo, GitHub sign-in, a first sync, and Insights → Generate.

Use a custom OAuth callback carefully

The callback URL registered with GitHub must match exactly what the server derives. On Vercel it is built from the production domain automatically; set `BASE_URL` only for a custom domain.

18.3 Environment variables

Variable	Required	Default / meaning
<code>DATABASE_URL</code>	yes	PostgreSQL connection string (<code>DATABASE_SSL=true</code> if the host requires TLS with a non-public CA)
<code>GITHUB_CLIENT_ID</code> , <code>GITHUB_CLIENT_SECRET</code>	prod	OAuth app credentials; without them GitHub sign-in is hidden and only the demo works
<code>JWT_SECRET</code> , <code>TOKEN_ENCRYPTION_KEY</code>	prod	Session signing and token encryption; production refuses to boot without them
<code>BASE_URL</code> , <code>FRONTEND_URL</code> , <code>GITHUB_CALLBACK_URL</code>	no	Derived from Vercel's production domain or <code>http://localhost:PORT</code>
<code>GITHUB_SCOPE</code>	no	<code>read:user user:email repo</code> ; use <code>public_repo</code> to limit to public repositories
<code>AI_PROVIDER</code> , <code>GEMINI_API_KEY</code> , <code>GEMINI_MODEL</code> , <code>OPENAI_API_KEY</code> , <code>OPENAI_MODEL</code>	no	Provider auto-selected from whichever key exists; <code>local</code> otherwise
<code>AI_DAILY_LIMIT</code>	no	20 paid reports / user / 24 h (0 = unlimited)
<code>CRON_SECRET</code>	for digests on Vercel	Enables <code>/api/cron/digest</code> ; Vercel sends it as a bearer token
<code>RESEND_API_KEY</code> , <code>DIGEST_FROM</code>	for e-mail	Slack needs nothing on the server
<code>DIGEST_SCHEDULER</code> , <code>DIGEST_HOUR_UTC</code>	no	In-process scheduler on/off (default on) and hour (default 7)
<code>DB_POOL_MAX</code> , <code>MIGRATE_ON_START</code>	no	Pool size (3 on Vercel, else 10); set <code>false</code> to skip migration checks once schema is current
<code>RATE_LIMIT_SCALE</code>	no	Multiplies every limit (tests use 100; leave 1 in production)
<code>DEMO_ENABLED</code>	no	<code>false</code> disables the demo workspace

18.4 Serverless versus long-running

Aspect	Serverless (Vercel)	Container
Boot	Lazy on first request; migrations under advisory lock; failed boot retried, 503 meanwhile	Migrate, then listen
DB connections	≤ 3 per instance, via Neon's pooler	Pool of 10
Digest trigger	Vercel Cron → HTTP endpoint	In-process timer + advisory lock
Rate limits / in-flight map	Per instance	Per process
Time limit	60 s per request (configured)	None, but outbound calls have timeouts
Static assets	Vercel CDN with immutable caching of <code>/assets</code>	Served by Express (<code>/assets</code> immutable for 1 year, <code>index.html</code> always revalidated)

18.5 Docker

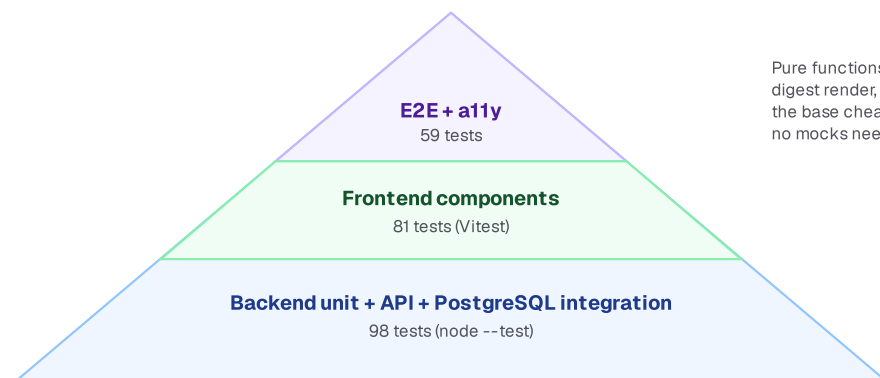
Multi-stage `Dockerfile`: (1) build the React app, (2) install production-only backend dependencies, (3) copy backend, migrations and the built frontend into a slim Node 22 image and run as the unprivileged `node` user, listening on 4000. `docker-compose.yml` adds PostgreSQL 16 with a health check for local use.

18.6 Running locally

```
# 1. PostgreSQL (or: docker compose up db)
# 2. Configure
cp .env.example .env          # set GITHUB_CLIENT_ID/SECRET, JWT_SECRET, TOKEN_ENCRYPTION_KEY
# 3. Backend
cd backend && npm ci && npm run dev      # http://localhost:4000, migrates on start
# 4. Frontend (separate terminal)
cd frontend && npm ci && npm run dev     # Vite dev server proxies /api
# Without any GitHub credentials, click "Explore the live demo".
```

19 Testing, quality gates and CI/CD

Real browser,
real server,
real database,
both themes



Pure functions (analytics,
digest render, reviews) make
the base cheap to test:
no mocks needed

Plus ESLint on both packages (React hooks compiler rules) and Dependabot.

Figure 21. Test pyramid with current counts.

19.1 Backend (98 tests, `node --test`, run serially against a real PostgreSQL)

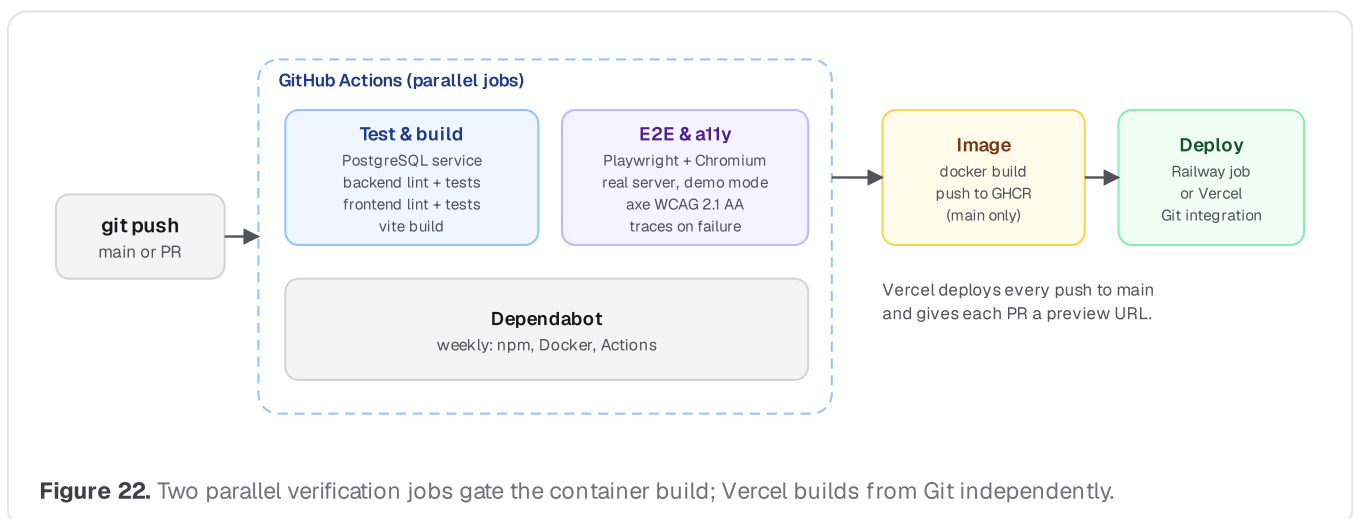
Suite (file)	What it proves
<code>analytics</code>	Time-zone bucketing, previous period, percentiles, streaks, lead-time and first-review histograms, waiting list, reviewer load, size buckets, data-quality rules
<code>reviews</code>	Review semantics: author excluded, PENDING excluded, bots never the first reviewer, ordering
<code>sync</code> + <code>utils</code>	<code>uniqueBy</code> ; configuration derivation (public URL, small pool on Vercel); validators
<code>integration</code> (PostgreSQL)	Migrations apply once and are safe concurrently; a pre-migration database is adopted; sync is incremental and prunes; an oversized delta falls back to a full resync; repository-list cache; vanished repositories; account deletion removes every trace and revokes the grant ; the demo account cannot be deleted; paid-AI daily quota; only 20 reports kept; an idle DB error does not crash; review backfill runs once; duplicate rows from shifting pagination are tolerated; the demo runs the real pipeline
<code>api</code>	Health (and degraded when the DB is down); protected routes require a session; profile returns no secrets; OAuth redirect carries a state cookie; callback rejects a mismatched state and, when valid, creates a session with an encrypted token; validation returns 400 before touching GitHub; JSON 404; public config advertises only what works; demo can be disabled; missing OAuth credentials give a readable redirect, not a 500
<code>security</code>	Forged <code>alg: none</code> tokens rejected; wrong issuer/algorithm rejected; every response has a request id and unsafe incoming ids are replaced; logs never include query strings; errors carry the request id
<code>reliability</code>	Upstream calls time out instead of hanging; a genuinely stalled server is cut off at the deadline; rate limits scale for tests yet bite by default
<code>team</code>	Totals equal the sum of repositories and people merge across them; unreadable repositories are reported, not fatal; default = analyzed repositories only; validation and the 5-first-sync cap; one user's repositories never visible to another; <code>mapLimit</code> preserves order within the concurrency limit

Suite (file)	What it proves
<code>digest</code> (unit)	<code>isDue</code> across weekdays, time zones and retries; rendering escapes HTML and Slack; links limited to https; subject and stat formatting
<code>digest.integration</code>	Exactly-once claim with concurrent runners; partial delivery status; cron auth (404 without secret, 401 wrong secret); the demo cannot send; the Slack URL is encrypted at rest and never returned
<code>ai</code>	Parser accepts fenced JSON and normalises; local fallback without a key; Gemini call parsed; provider failures surface as 502; prompt and local report contain the metrics
<code>serverless</code>	Boots on first request and migrates; demo works through the handler; a misconfigured deployment answers 503 and recovers; boot failure is retried

19.2 Frontend

- **81 unit/component tests** (Vitest + Testing Library, jsdom): formatting, chart data, signals (including the new review signals), `useAsync` (abort, stale values), `useInView`, UI kit, KPI grid (with and without review data), PR table (filter/search/pagination), repository comparison sorting, waiting list, reviewer card, Slack preview (links, bold, and "never turns text into markup"), delete-account dialog, landing sections.
- **59 end-to-end tests** (Playwright, real Chromium, real Express server, real PostgreSQL, built frontend, demo mode): sign-in, chart tooltips, URL-state round trips, command menu, theme, reduced-motion, mobile layout, the new Reviews/Team/Digest flows, and an **axe-core scan of every page in both themes**.
- **ESLint** on both packages, including React hooks and compiler rules.
- Screenshots (README and landing) are *generated by tests* from the running app, so documentation cannot drift from the product.

19.3 CI/CD



The workflow runs a PostgreSQL service container, installs, lints, tests and builds both packages, then runs the Playwright suite (uploading traces on failure). On `main` a Docker image is built and pushed to GitHub Container Registry and a Railway deploy job runs when configured. Dependabot opens weekly update PRs for npm, Docker and Actions.

Honest status

Actions are disabled by default on a fork and must be enabled by the repository owner; as of writing the pipeline has run locally in full (lint, 98 + 81 + 59 tests, build) but has not yet run on GitHub's runners.

20 API reference

All routes are under `/api`. JSON in and out; errors are `{"error": "...", "requestId": "..."}.` Authenticated routes need the `devpulse_token` cookie.

Method & path	Auth	Purpose	Key inputs → output
GET /health	no	Liveness + DB ping	→ <code>{status: ok degraded}</code> (503 when DB down)
GET /config	no	What this deployment supports	→ GitHub login / demo / AI / e-mail availability
GET /auth/github	no	Start OAuth	302 to GitHub
GET /auth/github/callback	no	Finish OAuth	<code>code</code> , <code>state</code> → Set-Cookie + 302
POST /auth/demo	no	Start demo session	204 + cookie
POST /auth/logout	no	Clear session	204
GET /user	yes	Current user	→ <code>{id, login, name, email, avatarUrl, isDemo}</code>
DELETE /account	yes	Delete everything, revoke grant	204
GET /repositories	yes	Accessible repositories + 30-day activity of analyzed ones	<code>refresh=true</code> → array
GET /repositories/:owner/:repo/commits	yes	Recent commits	<code>days</code> → array
GET /repositories/:owner/:repo/pulls	yes	Recent PRs with review fields	<code>days</code> → array incl. <code>isDraft</code> , <code>additions</code> , <code>firstReviewAt</code> , <code>reviewsKnown</code>
GET /analytics/summary	yes	Full per-repo summary	<code>repo</code> , <code>days</code> (1-90), <code>tzOffset</code> , <code>refresh</code> → summary (chapter 10)
GET /team/summary	yes	Multi-repo summary	<code>repos</code> (≤ 15), <code>days</code> , <code>tzOffset</code> → <code>{aggregate, repos, skipped}</code>
POST /ai/sprint-summary	yes	Generate and store a report	<code>{repo, days, tzOffset}</code> → 201 report
GET /ai/reports	yes	Last 10 reports	<code>repo</code> → array
GET /digest	yes	Digest settings	→ settings + channel availability + masked e-mail
PUT /digest	yes	Save settings	<code>repos</code> (≤ 10), <code>weekday</code> , <code>sendEmail</code> , <code>includeAi</code> , <code>enabled</code> , <code>slackWebhookUrl</code> (set / replace / "" to disconnect)
DELETE /digest	yes	Remove subscription	204

Method & path	Auth	Purpose	Key inputs → output
POST /digest/preview	yes	Render without sending	<code>repos</code> → subject, html, text, slack, skipped
POST /digest/send-test	yes	Send now to configured channels	→ per-channel result (502 if all failed)
GET POST /cron/digest	bearer secret	Run due digests	→ counts: considered, due, sent, failed, skipped, deferred

21 Limits, trade-offs and roadmap

21.1 Known limits

Limit	Consequence	Mitigation / honesty
History window 180 days (view $\leq$ 90 days)	No year-over-year views	Deliberate: bounded storage and cost
GitHub pagination: 1,000 commits, 500 PRs per repository	Very busy repositories are incomplete	Banner + previous-period comparison withheld
First 30 reviews per PR	Extremely long review threads under-count	Rare; reviews beyond 30 do not change "first review"
Review requests not visible	Cannot tell who a PR is waiting <i>on</i>	Waiting list shows age and author only
Per-instance rate limits and in-flight map	Weaker protection on serverless	Idempotent writes; Redis if abuse appears
Not real-time	Up to 2 minutes of cache; no webhooks	Refresh button; webhooks would need an always-on receiver
Heuristic thresholds for signals	Opinionated ratings	Visible and documented; no hidden score
Time-zone is the viewer's, not the team's	Cross-time-zone teams see different "busiest hours"	Per-person view is honest to the viewer
Digest runs once per day	Delivery within the hour is not guaranteed on Vercel Cron	Weekly cadence makes this irrelevant

21.2 Unverified in the development environment

Needs your credentials to confirm

Real GitHub OAuth and GraphQL responses, real Gemini/OpenAI calls, Resend e-mail and Slack webhook delivery, the Docker build, a Vercel deployment and the GitHub Actions run could not be exercised without secrets or network access. They are covered by unit/integration tests with injected fakes and by the demo pipeline, which implements the same interfaces, but first contact with the real services may reveal small differences (for example a field GitHub omits).

21.3 Deliberate non-goals

- No code analysis, no commit-message mining, no per-person "productivity score" or leaderboard framing.
- No issue tracker, CI-duration or deployment data (yet).
- No team accounts or shared workspaces: each user sees their own GitHub-authorized view.

21.4 Roadmap candidates

Idea	Value	Cost / risk
DORA metrics (deployment frequency, lead time for changes, change-failure rate, time to restore)	Standard language for engineering health	Needs deployment/release data (GitHub Deployments or Releases); definitions vary by team
GitHub webhooks	Real-time and fewer polling calls	Needs an always-on endpoint and signature verification
Review-request awareness	"Waiting on @alice" instead of "waiting"	Extra GraphQL fields and timeline events
Shared team workspaces / GitHub App	One install for an org; no per-user tokens	Admin approval, new auth model
Redis-backed rate limiting	Correct limits across serverless instances	Another service (free tier available)
Trend history beyond 180 days	Quarter/year views	Store daily aggregates instead of raw rows
Slack OAuth app	Channel picker instead of pasted webhooks	App review, more scopes

21.5 Is it "just GitHub in another tab"?

Partly, and the document should say so plainly. The activity charts and contributor lists reproduce what GitHub already shows and are included as context. The product earns its place through three things GitHub's repository Insights do not do: (1) **measure waiting** - time to first review and the live list of what nobody has looked at; (2) **show distribution** - how reviews and commits concentrate on people; (3) **push the answer** - a weekly digest and a team view across repositories. If a team does not care about review latency, DevPulse is a nice dashboard; if it does, it is a measurement GitHub does not provide.

22 Glossary and repository map

22.1 Glossary

Term	Meaning
Lead time / time to merge	Elapsed time from PR creation to merge
Time to first review	Elapsed time from PR creation to the first submitted review by a human other than the author
Waiting (for a first review)	An open, non-draft, non-bot PR whose reviews are known and contain no human review yet
Eligible PR	Non-draft, not bot-authored, <code>reviews_known = true</code>
Stale PR	Open for more than 14 days
Reviewer load / top reviewer share	Distribution of submitted reviews across people; share of the busiest
Previous period	Equal-length window immediately before the selected range
Truncated history	GitHub pagination ended before the window start; <code>history_from</code> records the oldest data we have
tzOffset	Minutes east of UTC for the viewer; used to bucket days and hours
In-flight sync	A sync already running for the same user and repository; concurrent callers await it
Advisory lock	A PostgreSQL application-level lock used to serialise migrations and the digest scheduler
Claim	The single UPDATE that lets exactly one digest runner process a subscription
Portfolio route	A page about the whole account (Repositories, Team, Digest) rather than one repository
Demo workspace	Shared generated account running through the real pipeline
Nearest-rank percentile	Percentile that returns an observed value: <code>sorted[ceil(p*n) - 1]</code>

22.2 Repository map

```
DevPulse/
├─ api/index.js          Vercel entry: createHandler({ config })
├─ vercel.json           build, function (60 s), rewrites, cache headers, daily cron
├─ Dockerfile, docker-compose.yml, railway.json
├─ .github/workflows/    CI/CD (test, e2e+a11y, image, deploy), Dependabot
├─ database/migrations/  001_init ... 005_digests (applied under advisory lock)
├─ backend/
│   ├─ server.js         long-running entry + digest scheduler
│   ├─ serverless.js     lazy-boot handler for Vercel
│   ├─ app.js            middleware order, static SPA, error handler
│   ├─ routes/index.js   createRouter(config): URL → controller, rate limits
│   ├─ controllers/      auth, user, repository, analytics, team, ai, digest
│   ├─ services/         github, sync, analytics, team, ai, digest(+Render), demoData, reviewUtil, token
│   ├─ middleware/       auth, errorHandler, requestLog
│   ├─ config/           env, db, migrate
│   ├─ utils/            crypto, http (timeouts), validate, httpError
│   └─ tests/            node --test suites (unit, API, PostgreSQL integration)
├─ frontend/
│   ├─ src/App.jsx, main.jsx
│   ├─ src/pages/        Landing, Overview, Repositories, Team, Pulls, Reviews, People, Insights, Digest, NotFound
│   ├─ src/state/        auth.jsx, workspace.jsx (URL-driven)
│   ├─ src/lib/          useAsync, signals, chartData, format, hooks, theme
│   ├─ src/services/api.js  fetch client, ApiError, tzOffset
│   ├─ src/components/   ui/, charts/, dashboard/, layout/, landing/, team/
│   ├─ src/styles/       index.css (tokens, motion), landing.css
│   ├─ e2e/              Playwright specs + screenshot/asset generators
│   └─ public/landing/   generated WebP screenshots
└─ docs/                ARCHITECTURE.md, DEPLOY_VERCEL.md, screenshots/, this guide
```

Document generated from the repository at the time of writing; diagrams are drawn directly from the code paths described. Where a statement depends on a third-party service's current terms or limits, it is flagged as such.